

A COMPLETE PROJECT GUIDE

Worker Safety (PPE) Monitoring System

From First Principles to Working Prototype

A beginner-friendly guide to the ideas, the architecture and the phase-by-phase build of a real-time computer-vision safety system — written so that anyone, with no prior background in computer vision, can understand what this project does, why it is built the way it is, and how to build it themselves.

Computer Vision · Edge AI · Real-Time Systems
Sixth edition · the project complete · 29 September 2026

Preface — How to Read This Book

This book has one job: to take you from knowing nothing about computer vision or real-time systems to a point where you understand, in real depth, what a Worker Safety (PPE) Monitoring System is, why every one of its pieces exists, and how to build it yourself. By the end, you should be able to explain the whole idea to a friend, a classmate, or an interviewer, in your own words, without leaning on jargon you don't actually understand.

No prior knowledge of artificial intelligence is assumed. If you can write basic code and you are willing to look a new word in the eye instead of skipping past it, you have everything you need to start. Every technical term is explained in plain language the first time it appears, usually with an everyday analogy before the formal definition, and every chapter that introduces new vocabulary ends with a short "Key Terms" box you can use as a quick-reference glossary later.

The book is organised in five parts:

- **Part I — Understanding the Problem.** Why this project needs to exist at all, how safety compliance is handled today, exactly what breaks, and who ends up using the system once it's built.
- **Part II — The Concepts You Need.** The largest part of the book. A guided tour through every idea the project rests on — how computers "see" images, how object detection and YOLO work, how tracking gives objects an identity across time, how live video streaming differs from playing a file, how models are made fast enough to run on ordinary hardware, and how the backend (databases, APIs, alerts) ties it together.
- **Part III — System Architecture.** The big picture: one diagram, one full walkthrough of a single violation's journey from camera to phone, and the reasoning behind the key design decisions.
- **Part IV — Building It.** A phase-by-phase implementation guide, from an empty repository to a working, benchmarked system, plus a week-by-week study plan. Each phase that has been built ends with an **As Built** section: what was actually built, what changed from the plan and why, the measured results, and the lessons.
- **Part V — Appendices.** A glossary, a metrics reference sheet, a prerequisites checklist, further reading, a short FAQ, and the build log.

This is a living book. The project it describes is being built phase by phase, and the book grows with it: Parts I–III explain the ideas and stay as first written, while the build notes in Part IV and the log in Appendix F record what happened when those ideas met real hardware and real data — including the places where the plan had to change. Where a build note and an earlier chapter disagree, the build note is the newer, measured answer.

The Full Plan

Part I — Understanding the Problem

- ✓ 1. Why Worker Safety Monitoring Matters
- ✓ 2. How Safety Compliance Works Today — and Where It Breaks
- ✓ 3. The Problem This Project Solves
- ✓ 4. Who Uses This System, and What Success Looks Like

Part II — The Concepts You Need

- ✓ 5. How a Computer "Sees": Images, Pixels and Video
- ✓ 6. Object Detection: Teaching a Computer to Find Things
- ✓ 7. Neural Networks and CNNs, Without the Maths Headache
- ✓ 8. YOLO: You Only Look Once
- ✓ 9. From Detection to Tracking: Giving Objects an Identity
- ✓ 10. Geometry Rules: Matching PPE to People and Spotting Zone Intrusion
- ✓ 11. Live Video Streaming: RTSP and Why Live Is Harder Than Recorded
- ✓ 12. Making Models Fast: ONNX, TensorRT and Quantization
- ✓ 13. The Backend: APIs, Databases, Queues and Alerts

Part III — System Architecture

- ✓ 14. The Big Picture: End-to-End Architecture
- ✓ 15. Life of a Violation: A Step-by-Step Walkthrough
- ✓ 16. Design Decisions and Trade-offs

Part IV — Building It: The Phase-Wise Guide

- ✓ 17. Phase 0 — Setup and Foundations (built)
- ✓ 18. Phase 1 — Data and Baseline Detector (built)
- ✓ 19. Phase 2 — Tracking and PPE Logic (built)
- ✓ 20. Phase 3 — Restricted Zones and Rules Engine (built)
- ✓ 21. Phase 4 — Multi-Stream Ingestion and Optimization (built)
- ✓ 22. Phase 5 — Alerts, Storage and Dashboard (built)
- ✓ 23. Phase 6 — Evaluation, Hardening and Packaging (built)
- ✓ 24. The Study Plan: What to Learn, Week by Week

Part V — Appendices

- ✓ A. Glossary of Terms
- ✓ B. Metrics Reference Sheet
- ✓ C. Prerequisites Checklist
- ✓ D. Further Reading
- ✓ E. Frequently Asked Questions
- ✓ F. Build Log
- ✓ G. Using the System

New in this edition (sixth, 29 September 2026): the project is complete. All six phases are built, checked and reviewed. Appendix G is the guide to using the system: one command for each everyday task, how to add your own cameras, zones and alerts, how to keep it accurate, and how to use the model on its own. **Fifth edition (28 September):** Chapter 23's Phase 6 build notes, and with them all six phases are built. Chapter 4's five numbers were measured end to end, on 260 test clips played as cameras. Three meet their targets and two fall short, and the notes say which, and why. The system now warns when a camera goes quiet. Docker Compose brings the whole system up on a clean machine. Appendix B now gives the measured value beside each target. Last, the system met real video from six other sites (free stock clips): four of six violations caught, and three kinds of mistake the photo clips had hardly shown. Then the detector was taught that caps and hats are not helmets: people in hats judged "helmet worn" fell from 27% to 2.5%, in a second attempt after the first was rejected. **Fourth edition:** Chapter 22's build notes: events live in PostgreSQL with their evidence images, alerts reach a phone through Telegram with the picture three to four seconds after the event, and a dashboard shows the cameras, the counts and the events, with a false-alarm button. **Third edition:** Chapter 23 reports the fine-tuning. The detector, trained further on three more public datasets, finds the navy, brown and white helmets it used to miss on real workshop video (navy: 41% of frames before, 96% after). It is level or better almost everywhere else; the exception, vests, is set out plainly. The overfitting check flagged it by a small margin, and the chapter says why it was accepted anyway. Chapter 21 adds a speed-up: one Mac now watches eight cameras at 10 frames a second. The second edition added the build notes for Phases 0–4 (Chapters 17–21), with every number measured on a MacBook Air. Appendix F is the dated log of the build.

PART I

Understanding the Problem

Before writing a line of code, it's worth being precise about what is broken today, for whom, and why. This part builds that foundation.

CHAPTER 1

Why Worker Safety Monitoring Matters

Picture a construction site on an ordinary morning. Cranes swing steel overhead. Trucks reverse between stacks of material. Workers move between scaffolding, open trenches, and areas where heavy machinery is turning. Two pieces of equipment are so basic that they barely register as "technology" at all: a hard hat, and a vest bright enough to catch a driver's eye. Together, these two items — collectively part of what the safety world calls **PPE**, or **Personal Protective Equipment** — are among the cheapest, oldest, and most effective interventions in industrial safety. And yet, getting every worker to wear them, every hour of every shift, turns out to be a surprisingly hard problem to solve at scale.

It is worth pausing on why this matters as much as it does. According to the International Labour Organization (ILO), work kills close to three million people a year worldwide when fatal accidents and long-term occupational disease are counted together — roughly 330,000 of those deaths come from accidents on the job itself, not disease. The ILO names construction as one of the four most hazardous sectors, alongside agriculture, forestry and fishing, and manufacturing; together they account for around 200,000 fatal injuries a year — about 63 per cent of all fatal occupational injuries worldwide. These are not abstract numbers; they are the reason safety regulation exists at all, and the reason a site without visible, enforced safety rules is not just breaking a rule but carrying real risk.

In the United States, the Occupational Safety and Health Administration (OSHA) groups the most common causes of construction deaths into what it calls the **Fatal Four**:

- **Falls** — consistently the single largest cause. In 2023, falls to a lower level alone killed 421 of the 1,075 US construction workers who died on the job.
- **Struck-by incidents** — a worker hit by a vehicle, falling object, or swinging load.
- **Caught-in or caught-between** — a worker trapped or crushed by equipment, material, or a collapsing structure.
- **Electrocution** — contact with live electrical equipment or power lines.

In OSHA's published figures, these four causes together account for roughly six in every ten construction deaths. PPE does not eliminate any of them on its own, but helmets and vests blunt the most common of them directly: a helmet absorbs the blow from a dropped tool or falling debris — the classic struck-by injury — and reduces head injury in slips and falls, while a high-visibility vest gives a truck driver or excavator operator the extra moment of recognition needed to stop before a worker is struck. This is why helmets and vests are usually the very first thing a safety inspector checks, and why construction sites, ports, oil and gas facilities, and factory floors make them mandatory in designated areas.

PPE only softens the blow once something has already gone wrong, though, and for some hazards no amount of it is enough. A hard hat will not save someone standing under a

two-tonne load when a sling fails, inside the swing radius of an excavator, at the edge of an unsupported trench, or next to exposed live equipment. For those hazards — which map directly onto struck-by, caught-in/between and electrocution — the only effective protection is distance: keeping people out of the area entirely. That is why this project watches for two things, not one: whether workers are wearing their PPE, and whether anyone has entered a **restricted zone** they are supposed to stay out of.

The trouble is that a rule written on a signboard at the site entrance does not enforce itself. A worker who is hot, in a hurry, or simply forgetful will take a helmet off for five minutes, or duck under a barrier tape to save a long walk round — and five minutes is exactly long enough for something to go wrong. Making sure that does not happen, across a site with dozens or hundreds of workers moving in and out of different zones all day, is fundamentally a monitoring problem: someone, or something, has to be watching, continuously, and has to notice quickly enough to matter. The next chapter looks at how that monitoring is done today, and why the people doing it are fighting a losing battle against sheer scale.

Sources: International Labour Organization, "Nearly 3 million people die from work-related accidents and diseases" (2023); U.S. Bureau of Labor Statistics construction fatality data for 2023, as reported in OSHA's fall-prevention material; OSHA, Fatal Four hazard categories.

Key Terms Introduced in This Chapter

PPE (Personal Protective Equipment) — Equipment worn to reduce exposure to a hazard — in this project, specifically helmets and high-visibility vests.

Fatal Four — OSHA's name for the four hazard categories (falls, struck-by, caught-in/between, electrocution) behind most US construction deaths.

Restricted zone — An area people must stay out of — always, or at certain times — because no PPE makes it safe, such as the space under a suspended load.

CHAPTER 2

How Safety Compliance Works Today — and Where It Breaks

Almost every regulated worksite already has a system for checking PPE compliance. It usually looks something like this: a safety officer, sometimes called an EHS (Environment, Health and Safety) coordinator, walks a fixed route through the site on a schedule — perhaps once every hour, perhaps a few times a shift. They carry a clipboard or a phone app, note down anything they see (a worker without a helmet near the east scaffold, a contractor inside a taped-off excavation), and follow up, either on the spot or in the next toolbox talk. Alongside this, most sites also run CCTV — closed-circuit television — with cameras covering entrances, cranes, storage yards and hazardous zones.

Both halves of this system are useful, and neither is sufficient on its own. The patrol model is limited by a simple fact of physics: one person can only be in one place at a time. A safety officer who checks the east scaffold at 9:00 a.m. has no idea what happens there at 9:15, or at the west trench at 9:00, or at any of the dozen other spots on a mid-sized site that need watching. The CCTV half of the system solves the coverage problem — the cameras see far more of the site than any one person can — but introduces a different one: nobody is watching all the feeds, all the time. On a site with even a modest twenty or thirty cameras, that is more screens than any control-room operator can meaningfully track. Research on vigilance — a person's ability to keep noticing rare events in a stream of mostly uneventful input — has found since the 1940s that people start missing more of those events within the first fifteen to thirty minutes of continuous watching, however conscientious they are. This is not a character flaw; it is how human attention works, and it is exactly the kind of task computers are good at instead.

The practical result is that CCTV footage on most sites is watched in only one circumstance: after something has already gone wrong, when someone rewinds the tape to understand what happened. That is valuable for investigation and for insurance claims, but it is the opposite of prevention. By the time anyone reviews the footage, the worker has already been struck, or already fallen, or already walked away unharmed but with the same exposed risk still there for the next shift.

The obvious next idea is to point an off-the-shelf AI model at the camera feed and let it flag violations automatically. In practice, a naive version of this idea runs into trouble fast. A model that looks at each video frame in isolation, with no memory of what it saw a moment ago, will typically re-detect the same ongoing violation on every single frame — at, say, 15 frames per second, one worker without a helmet for ten seconds becomes 150 separate alerts. A model with no concept of "zone" cannot tell the difference between a worker standing just outside a fenced-off excavation and one standing inside it. And a model that was only ever tested on a handful of clean, well-lit sample images tends to degrade badly

on a real site, where cameras are mounted at awkward angles, half a worker is hidden behind a pillar, and the light changes completely between 7 a.m. and noon. The predictable result is a flood of low-quality alerts that safety staff quickly learn to ignore — which is arguably worse than having no automated system at all, because it burns the very trust the system needs to be useful.

None of this means automation is the wrong idea — it means the naive version of it is the wrong idea. Getting it right requires solving several distinct engineering problems at once, not just "running a detector on a video." The next chapter lays out exactly what those problems are, because they define everything the rest of this book builds toward.

Key Terms Introduced in This Chapter

CCTV — Closed-Circuit Television — a private camera network, as opposed to a public broadcast.

Vigilance decrement — The well-documented drop in a person's ability to detect rare events the longer they continuously watch a mostly uneventful feed.

False positive / false alarm — A case where the system reports a violation that did not actually happen.

CHAPTER 3

The Problem This Project Solves

Strip away the specifics of cameras and construction sites for a moment, and the problem this project solves can be stated in a single sentence: **watch multiple live video streams, notice when a worker is missing required PPE or has entered a restricted zone, and tell the right person within seconds — with proof — without crying wolf and without staying silent when it matters.** Every phrase in that sentence is doing real work, and unpacking it reveals four separate sub-problems that a naive "just run a detector" approach, as described in the last chapter, fails to solve.

The Detection Problem

The first and most obvious sub-problem is telling a helmet from a bare head, and a high-visibility vest from an ordinary shirt, reliably enough to trust. This sounds trivial until you consider real site conditions: a worker filmed from directly above by a mounted camera looks nothing like the same worker filmed from the side; a bright orange t-shirt can be mistaken for a vest; a cap can be mistaken for a helmet at low resolution; and half of the relevant person may simply be hidden behind a pillar, a vehicle, or another worker. Spotting the equipment is not enough on its own, either: a helmet sitting on a railing next to a worker, or carried in their hand, is in the picture but not on anyone's head. For every person, the system has to decide whether each item is actually being *worn*. Part II solves this in two steps — object detection to find people, helmets and vests (the chapters on detection, neural networks and YOLO), then simple geometry to match each item to the person wearing it (Chapter 10).

The Identity-and-Time Problem

The second sub-problem, introduced briefly in the last chapter, is that a single photograph in isolation cannot tell the difference between "this violation just started" and "this violation has been running for ten minutes and has already been reported ninety times." The system needs to recognise that the person in this frame is the same person it saw two seconds ago, so it can count one continuous violation as one **event** rather than one event per frame — and so it can ignore a violation that flickers into a single frame and out again. This requires giving each detected person a persistent identity across time — the subject of Part II's chapter on tracking.

The Geography Problem

The third sub-problem is that "who" is only half the picture — the system also needs to reason about "where." A restricted zone (say, the area directly under a suspended load, or inside an active excavation) is not a fixed part of the video frame; it is a region a safety

officer defines for each specific camera, and the system needs a way to let them draw it, store it, and then continuously check whether a tracked person's position falls inside it. This is a geometry problem as much as a vision problem, covered in Part II's chapter on zone rules.

The Scale-and-Trust Problem

The fourth sub-problem is the one that determines whether any of the above actually gets used in practice. A real site does not have one camera; it has many, and the computer running this system is not a data-centre server — it is whatever modest GPU or CPU box the budget allows. The system has to process several live streams at once, in close to real time, without the cost of the hardware becoming a blocker. And critically, it has to be trustworthy in both directions. If its alerts are wrong one time in three, people will likely mute it within a week, no matter how technically impressive its underlying model is; if it stays silent while a real violation carries on, it has failed at the one job it exists to do. This is why the false-alarm rate and the missed-violation rate are treated, throughout this book, as just as important as raw detection accuracy — a point Chapter 4 returns to when it defines what "success" actually means for this project.

The Objective, in One Paragraph

Objective: Build and benchmark an end-to-end, near-real-time system that watches several live CCTV streams on a single affordable machine; detects workers who are not wearing a helmet or high-visibility vest, and people who enter a restricted zone; confirms each case as one trustworthy event rather than a flood of duplicates; and alerts the safety officer within seconds — a phone message with a snapshot as evidence — while logging every event to a dashboard where it can be reviewed, confirmed or rejected, and fed back to improve the model.

Each clause of that objective maps onto one of the four sub-problems. *Detects workers who are not wearing* their PPE is the Detection Problem; *confirms each case as one event* is the Identity-and-Time Problem; *restricted zone* is the Geography Problem; and *several streams on a single affordable machine*, together with *trustworthy*, is the Scale-and-Trust Problem. Chapter 4 turns vague words like "within seconds" and "trustworthy" into numbers you can measure. Everything from Part II onward is, in one way or another, in service of this paragraph.

Key Terms Introduced in This Chapter

Event — One confirmed violation by one tracked person, counted once no matter how many video frames it spans.

Near-real-time — Fast enough that the response still matters — here, seconds, not minutes.

CHAPTER 4

Who Uses This System, and What Success Looks Like

It's easy to design a system that is technically impressive and practically useless, because nobody stopped to ask who would actually use it and what they need from it. This chapter fixes that by naming the people on the other end of this system, walking through a day in their working life once the system exists, drawing a clear line around what the system does and doesn't do, and then defining — in plain words and concrete numbers — what counts as success.

The People Involved

- **The Site Safety Officer / EHS Coordinator** — the primary user. This is the person whose phone buzzes when a violation is confirmed, who reviews the dashboard, and who marks alerts as genuine or false so the system keeps improving.
- **The Site or Project Manager** — cares less about any single alert and more about trends: is compliance improving week over week, which zones and shifts generate the most violations, and is the evidence there for weekly safety reports.
- **The Auditor or Compliance Reviewer** — for insurance, certification, or client audits, this person needs an evidence trail: a timestamped photo showing what happened, where, and when, not just a logged number.
- **You, the builder** — if you are working through this project to learn, this system doubles as a proof of applied skill across several in-demand disciplines at once: computer vision, real-time systems, and backend engineering.

A Day in the Life, With the System Running

It's mid-morning, and a worker near a stairwell takes off their helmet to wipe sweat from their forehead, then sets it down on a railing instead of putting it back on. The system sees a helmet in the frame — but not on anyone's head. A single frame proves nothing (a bad angle or a momentary glitch could cause it), so the system waits until the violation has lasted a couple of seconds; a moment later, the safety officer's phone receives a message: a snapshot of the worker, the camera name, the zone, and the time. They walk over, the worker puts the helmet back on, and the moment is corrected before it becomes an incident rather than after. Later that day, the officer reviews the alert on the dashboard alongside a dozen others from the shift and tags a couple as false alarms — perhaps one where a worker's vest was hidden behind a handrail, so the system wrongly concluded it was missing. Those tagged examples quietly become the raw material for improving the model the next time it's retrained. Over weeks, this loop of alert, confirm-or-reject, and retrain is what turns a reasonable first version of the system into a genuinely reliable one.

What the System Does — and Deliberately Doesn't

A clear objective needs clear edges. Being able to say what a system *won't* do is as useful, when explaining it to someone else, as saying what it will.

In scope

- Finding people, helmets and high-visibility vests in live video, and deciding for each person whether each item is actually being worn.
- Restricted zones drawn per camera by the safety officer, with simple rules such as the hours a zone is active.
- Several cameras at once (the target is about eight) on one machine, in near real time.
- Alerts carrying a snapshot, camera, zone and time — by Telegram and email, with WhatsApp optional — plus a dashboard to review, confirm or reject them.
- Storing every event and every piece of officer feedback, for reports, audits and retraining.

Out of scope, for this version

- Identifying who a worker is. There is no face recognition; the system only follows anonymous track numbers.
- Other PPE such as gloves, goggles, harnesses or safety boots. The same approach extends to them later, but each needs its own training data.
- Detecting falls, fire, or unsafe machine operation — different problems that need different models.
- Acting on its own. The system alerts a person; it doesn't lock gates or stop machinery, and it doesn't replace the safety officer. It extends their eyes, while decisions and follow-up stay with people.

What "Success" Actually Means

This project measures success with five numbers. Read as raw terms, they can sound like abstract engineering targets; read in terms of the people above, each one answers a very concrete question — and each comes with a starting target to test against:

Metric	The Human Question It Answers	Starting Target
mAP@50	When a person, helmet or vest is in view, how reliably does the model find it and put the box in the right place? (Explained fully in Chapter 6.)	0.85 or higher, on scenes the model never trained on
Throughput	How many cameras can one machine watch at once, on hardware we can actually afford?	8 streams at 10+ frames per second each on one mid-range GPU

Metric	The Human Question It Answers	Starting Target
Alert latency	How long between a violation starting on camera and a human being told about it?	5 seconds or less, including the deliberate 2–3 second wait
Missed-violation rate	When a real violation happens and lasts, how often does the system stay silent?	At least 90% of real, sustained violations produce an alert
False-alarm rate	How often does the system cry wolf — and will people still be paying attention to it in a month?	At least 90% of alerts are real violations

These are starting targets, not promises: Phase 6 (Chapter 23) measures each one on real footage and replaces them with your own numbers. Appendix B collects them with notes on how to measure each.

Notice that only the first number is about the AI model itself; the other four describe the whole system as the safety officer experiences it. A great model inside a badly built pipeline still fails the objective. And the last two deserve special emphasis, because they pull against each other. Make the system more eager and it misses less but cries wolf more; make it more cautious and the reverse happens. A system that stays silent during a real violation has failed at its core job, and one that cries wolf too often gets muted — and a muted alert protects nobody. Naive projects tend to chase detection accuracy and forget the false-alarm rate. Every design decision in Part III and every phase in Part IV is, in some way, about keeping both low at the same time.

With the problem now fully in view — what it is, who it's for, and what "solved" means — the rest of this book turns to how it's actually solved. Part II starts from the most basic question of all: what does it even mean for a computer to "look at" an image?

Key Terms Introduced in This Chapter

Throughput — How much video the system can keep up with — here, number of streams × frames per second on one machine.

Alert latency — The time from a violation starting on camera to the alert reaching a person.

Missed-violation rate / false-alarm rate — The two ways an alert system can be wrong: staying silent about a real violation, or alerting about one that didn't happen. Measured per event, not per frame.

PART II

The Concepts You Need

Nine chapters, each introducing one idea the project rests on — from pixels to packet loss. No prior computer-vision background assumed; every term is defined the moment it first appears.

CHAPTER 5

How a Computer "Sees": Images, Pixels and Video

Everything in this book eventually rests on one basic fact: a computer does not see a photograph the way you do. It has no notion of "a hard hat" or "a person." What it receives is a grid of numbers — nothing more. Understanding exactly what that grid looks like is the first step toward understanding everything that happens to it afterward.

Think of a digital image as a very fine mosaic: a rectangular grid of tiny colored tiles called **pixels** (short for "picture elements"). A common camera frame might be 1920 pixels wide and 1080 pixels tall — a resolution written as 1920×1080, or "1080p" — which means the image is a grid of just over two million tiles. Each tile holds a color, and a color is itself stored as three numbers: how much red, how much green, and how much blue light it contains, each on a scale from 0 (none) to 255 (maximum). This is the **RGB** model, and it means a single color image is really three overlaid grids of numbers — three **channels** — stacked together. A grayscale image, by contrast, needs only one channel, since each pixel is just a single brightness value.

In code, this grid of numbers is usually represented as an **array** — a structured block of numbers a program can index into and manipulate — with a shape described as (height, width, channels). A 1080p color frame is therefore an array of shape (1080, 1920, 3): 1080 rows, 1920 columns, 3 color channels per pixel, for a little over six million individual numbers describing one still image.

EXAMPLE — reading an image as a grid of numbers (OpenCV)

```
import cv2

frame = cv2.imread("worker.jpg")
print(frame.shape)      # e.g. (1080, 1920, 3)
print(frame[500, 800]) # the [B, G, R] values of one pixel
```

Video simply adds a fourth dimension: time. A video is a sequence of individual images, called **frames**, played back quickly enough that human eyes perceive continuous motion rather than a slideshow. How quickly is quickly enough is measured in **frames per second (FPS)** — cinema traditionally uses 24 FPS, broadcast television around 25–30 FPS, and a security camera might run anywhere from 10 to 30 FPS depending on how it is configured. For this project, a live camera feed is nothing more than a continuous stream of these still-image arrays, arriving one after another, each of which the system will need to examine in a fraction of a second before the next one arrives.

This framing matters because it demystifies everything that follows. Object detection, in the next chapter, is not some mysterious act of "understanding" a picture — it is a very sophisticated set of mathematical operations performed on a grid of numbers, trained to

produce an output ("there is very likely a helmet-shaped pattern of numbers around row 200, column 850") that we then translate back into human-meaningful language.

Key Terms Introduced in This Chapter

Pixel — The smallest single element of a digital image — one tile in the mosaic.

Resolution — The image's grid size, given as width × height (e.g. 1920×1080).

RGB — A color model that stores each pixel as three numbers: red, green and blue intensity.

Channel — One of the separate numeric grids that make up an image — 3 for color (RGB), 1 for grayscale.

Frame — One still image within a video sequence.

FPS (Frames Per Second) — How many frames a video shows, or a camera captures, each second.

CHAPTER 6

Object Detection: Teaching a Computer to Find Things

With images understood as grids of numbers, the next question is how a computer finds specific things within that grid — a person, a helmet, a vest. This task has several close cousins that are worth telling apart before going further, because the distinction shapes everything downstream. **Image classification** answers "what is the single main subject of this whole image?" — useful for sorting photos, but useless here, since a single frame usually contains several people and objects at once. **Object detection**, the task this project actually needs, answers a harder question: "what objects are in this image, and exactly where is each one?" A third cousin, **segmentation**, goes further still, tracing the precise outline of each object pixel by pixel rather than just a rectangle around it — powerful, but usually unnecessary overhead for a task like this one.

An object detector's output, for a single image, is a list of detections, and each detection has three parts. First, a **bounding box**: the rectangle that tightly encloses the object, typically described by the pixel coordinates of its corners (`x_min`, `y_min`, `x_max`, `y_max`). Second, a **class label**: which category the object belongs to, drawn from a fixed list the model was trained on — in this project, categories like "person," "helmet" and "vest." Third, a **confidence score**: a number between 0 and 1 representing how sure the model is that this detection is correct. A detection of "helmet, confidence 0.94" is a strong signal; "helmet, confidence 0.31" is a weak guess the system will typically discard.

Once a model produces boxes, a natural next question is: how do we grade whether a predicted box is actually correct, compared to a box a human labeled by hand as the ground truth? The standard answer is a measurement called **Intersection over Union (IoU)**. Picture the predicted box and the true box as two overlapping rectangles. IoU is simply the area where they overlap, divided by the total area the two boxes cover between them:

EXAMPLE — computing Intersection over Union

```
def iou(box_a, box_b):
    # each box is (x_min, y_min, x_max, y_max)
    ix1 = max(box_a[0], box_b[0])
    iy1 = max(box_a[1], box_b[1])
    ix2 = min(box_a[2], box_b[2])
    iy2 = min(box_a[3], box_b[3])

    inter_area = max(0, ix2 - ix1) * max(0, iy2 - iy1)
    area_a = (box_a[2] - box_a[0]) * (box_a[3] - box_a[1])
    area_b = (box_b[2] - box_b[0]) * (box_b[3] - box_b[1])

    return inter_area / (area_a + area_b - inter_area)
```

A perfect match scores an IoU of 1.0; two boxes that don't touch at all score 0. A common convention, used throughout this project, is to call a detection "correct" if its IoU against the true box is at least 0.5 — which is exactly what the "50" in the **mAP@50** metric from Chapter 4 refers to: mean Average Precision, measured at an IoU threshold of 0.5. Two other terms round out how detectors are graded: **precision** asks, of everything the model flagged, how much was actually correct; **recall** asks, of everything that was actually there, how much did the model catch. A detector can trivially get perfect recall by flagging everything in sight (at the cost of precision), or perfect precision by only flagging things it's extremely sure about (at the cost of recall) — mAP is, roughly, a single number that captures how well a model balances the two across every confidence threshold at once.

One more piece completes the picture: it is common for a detector to fire multiple overlapping boxes around the same real object (three or four slightly different guesses at "where exactly is this helmet"). A cleanup step called **Non-Maximum Suppression (NMS)** resolves this by keeping only the highest-confidence box in any cluster of heavily overlapping boxes and discarding the rest — you'll see this term again in the next chapter, since it's built into how YOLO produces its final output.

Key Terms Introduced in This Chapter

Object detection — Finding what objects are in an image and where each one is located.

Bounding box — The rectangle that encloses a detected object.

Confidence score — A 0-1 number expressing how sure the model is about a detection.

IoU (Intersection over Union) — A measure of overlap between two boxes, used to grade whether a detection matches the truth.

Precision / Recall — Precision: how much of what was flagged is correct. Recall: how much of what's really there was caught.

mAP (mean Average Precision) — A single score summarizing detection accuracy across confidence levels, at a given IoU threshold (e.g. mAP@50).

NMS (Non-Maximum Suppression) — A cleanup step that collapses duplicate overlapping boxes down to one per real object.

CHAPTER 7

Neural Networks and CNNs, Without the Maths Headache

Object detectors like YOLO are built out of **neural networks**, and it's worth building an honest intuition for what that phrase actually means before treating it as a black box. Strip away the biological metaphor in the name, and a neural network is a large mathematical function made of many small, simple pieces stacked together, whose behavior is shaped entirely by adjustable numbers learned from data, rather than hand-written by a programmer.

The smallest piece is often called a **neuron** (or unit). Each one does something unremarkable on its own: it takes a handful of numbers in, multiplies each by an adjustable number called a **weight**, adds them up, and passes the result through a simple function that decides how strongly to "fire." What makes networks powerful is scale and structure — thousands of these units are arranged into **layers**, each layer's output feeding the next layer's input, so that simple pieces combine into something that can represent very complicated patterns. Early layers tend to respond to simple things — an edge, a change in brightness, a patch of a particular color. Deeper layers combine those simple responses into more abstract ones — a curve, a corner, eventually a shape recognizable as "the rim of a helmet" or "a shoulder strap."

How does a network end up with the right weights? Through **training**: showing it a large number of labeled examples (an image, and the correct answer for that image), letting it guess, measuring how wrong the guess was with a **loss** function, and then nudging every weight very slightly in the direction that would have made the guess less wrong. This nudging process is called **backpropagation** combined with **gradient descent**, and one full pass through the entire training dataset is called an **epoch**. Repeat this process — guess, measure error, nudge weights — over enough examples and enough epochs, and the network's weights gradually settle into values that produce genuinely useful answers on images it has never seen before. None of this book requires you to derive the mathematics of gradient descent by hand; what matters is the shape of the idea: learning here means adjusting numbers to reduce measured error, repeatedly, automatically.

For images specifically, a particular architecture called a **Convolutional Neural Network (CNN)** dominates, and it's worth understanding why. A plain neural network that looked at every pixel independently would need an enormous number of weights and would have no notion that nearby pixels are related. A CNN instead uses a small, reusable pattern-detector called a **filter** (or kernel) — commonly a 3×3 or 5×5 grid of weights — and slides it across the entire image, computing a small calculation at every position. Think of it like a stencil sized to catch a specific local pattern, such as a diagonal edge, dragged systematically over every patch of the photograph; wherever the pattern matches strongly,

that filter's output lights up. Because the same small filter is reused at every position (a property called **parameter sharing**), a CNN needs far fewer weights than a plain network would, and because filters scan local neighborhoods, the network naturally understands that pixels close to each other are related in a way that distant pixels are not — precisely the kind of structure a photograph actually has.

EXAMPLE — defining one convolutional layer (PyTorch)

```
import torch.nn as nn

# A single convolutional layer:
# in_channels=3 -> input has 3 color channels (RGB)
# out_channels=16 -> learn 16 different 3x3 filters
# kernel_size=3 -> each filter is a 3x3 grid of weights
conv_layer = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3)
```

A real CNN stacks dozens of these convolutional layers, interleaved with steps that shrink the image (so deeper layers see a larger effective area with less computation) and steps that combine channels together. By the time information reaches the last few layers of a modern detector, the network isn't reasoning about individual pixels at all anymore — it's reasoning about rich, learned patterns that correspond, in aggregate, to the objects this book cares about. That reasoning, purpose-built for speed, is what the next chapter's subject — YOLO — turns into finished bounding boxes.

Key Terms Introduced in This Chapter

Neural network — A large function built from many simple adjustable units, whose behavior is learned from data rather than hand-programmed.

Weight — An adjustable number that scales one input inside the network; learning means finding good values for these.

Layer — A group of units that all process the previous layer's output together.

Training / loss / epoch — Training: showing labeled examples and adjusting weights to reduce error. Loss: the measured error. Epoch: one full pass through the training data.

CNN (Convolutional Neural Network) — A network architecture built around small, reusable filters that scan across an image, well suited to visual data.

Filter / kernel — A small grid of weights that slides across an image, detecting one local pattern wherever it appears.

CHAPTER 8

YOLO: You Only Look Once

Object detection did not start out fast. The influential detectors that came before YOLO worked in two separate stages: first, a slow scan of the image proposing thousands of candidate regions that might contain something interesting; second, a separate classification step examining each candidate individually to decide what it was. This two-stage approach could be accurate, but running a full classification pass over thousands of candidate regions, for every single frame, was far too slow for anything resembling live video.

YOLO — **You Only Look Once**, first introduced by Joseph Redmon and colleagues in 2015 — reframed the entire problem. Instead of proposing regions and then classifying them separately, YOLO treats detection as a single regression problem, solved in one forward pass through one network: look at the whole image once, and directly output every bounding box, class, and confidence score at the same time. This single-pass design is the reason YOLO models remain, generation after generation, among the fastest usable object detectors available, which is exactly why this project (and the large majority of real-time computer vision projects) build on it rather than on a two-stage alternative.

The mechanics, simplified, work roughly like this. YOLO divides the image into a grid of cells. Each cell is made responsible for predicting any object whose center falls inside it, producing a set of candidate boxes together with a confidence score and a class probability for each. Because many cells and many candidate boxes will fire around the same real object, the network applies Non-Maximum Suppression (from Chapter 6) as a final cleanup step, collapsing overlapping duplicates down to one confident box per real object. The details of exactly how boxes are proposed have changed considerably across YOLO's many versions — from fixed-size "anchor boxes" in earlier versions to more flexible anchor-free designs in later ones — but the core idea, one pass, one network, boxes and classes together, has remained constant.

This project uses the Ultralytics implementation of YOLO, which packages a modern YOLO architecture behind a very small, approachable Python interface. Two features of this library matter a great deal for how the project is actually built. The first is **pretrained weights**: rather than starting from random weights and training on millions of images from scratch, you start from a model already trained on a large general-purpose dataset (which already knows what edges, textures, and general "person-shaped" patterns look like) and continue training it on a much smaller dataset of your own. This is called **transfer learning**, and the retraining step itself is called **fine-tuning** — it is dramatically faster and needs far less data than training from zero, which is exactly what Part IV's Phase 1 will do with a PPE-specific dataset.

EXAMPLE — running a pretrained YOLO detector

```
from ultralytics import YOLO

# Load a small, pretrained YOLO model
model = YOLO("yolov8n.pt")

# Run detection on a single image
results = model("site_camera_frame.jpg")
for box in results[0].boxes:
    print(box.cls, box.conf, box.xyxy)  # class, confidence, box coordinates
```

The second feature worth knowing about now, even though its full purpose only becomes clear in Chapter 9, is that the same library also supports feeding its detections straight into a tracker with a single extra argument. Detection alone, however accurate, only ever answers "what is in this one frame." The next chapter explains why that isn't enough, and what has to be added to turn a string of independent detections into a coherent understanding of a specific person moving through a scene over time.

Key Terms Introduced in This Chapter

Two-stage detector — An older detection approach: propose candidate regions first, classify them separately second. Accurate, but slow.

Single-stage detector (YOLO) — A detection approach that predicts all boxes and classes in one forward pass through one network — the reason YOLO is fast.

Grid cell / anchor box — The spatial unit YOLO divides an image into, and the template shapes it proposes boxes from.

Pretrained weights — Weights already learned from a large general dataset, used as a starting point rather than starting from random values.

Transfer learning / fine-tuning — Continuing to train a pretrained model on a smaller, task-specific dataset instead of training from scratch.

CHAPTER 9

From Detection to Tracking: Giving Objects an Identity

Chapter 3 introduced the identity-and-time problem: a detector examining frames one at a time has no way of knowing that the person it just found is the same person it found a tenth of a second earlier. Solving that is the job of **multi-object tracking (MOT)** — the process of assigning each detected object a persistent **track ID** that stays the same for as long as that object remains visible, frame after frame.

At its core, tracking is a matching problem, repeated every frame: given the set of tracks the system already knows about (each with a last-known position and, importantly, a velocity — the direction and speed it was last moving in) and the new set of detections the detector just produced for this frame, which detection belongs to which existing track? A good tracker doesn't just compare raw positions; it first **predicts** where each existing track should be in this new frame, based on its last known velocity, using an algorithm called a **Kalman filter**. Think of it as the same kind of estimate you make watching a ball thrown across a room: you don't need to see it in a given instant to have a strong guess where it is, based on where it just was and how fast it was moving. The tracker then compares each predicted position against the actual new detections — typically using the IoU measure from Chapter 6 — and solves an assignment problem (efficiently handled by an algorithm called the **Hungarian algorithm**) to find the best overall pairing between predicted tracks and real detections. A matched detection updates and continues its track; an unmatched detection starts a new one; a track with no matching detection for too many frames in a row is considered lost.

This project specifically uses **ByteTrack**, introduced by Zhang and colleagues in 2021, because it solves a problem that trips up simpler trackers on exactly the kind of footage this project deals with: occlusion. On a busy site, a worker is constantly walking behind pillars, vehicles, or other workers, and during those moments the detector's confidence in that person often drops — sometimes low enough that a simpler tracker would discard the detection entirely and lose the track. ByteTrack's key insight is refreshingly simple: don't throw away low-confidence detections immediately. Match high-confidence detections to tracks first, as usual, but then make a second matching pass using the low-confidence, previously-discarded detections against any tracks still unmatched. A person who is 40% visible behind a pillar is exactly the case a low-confidence detection was probably still describing correctly — and recovering it means the track survives the occlusion instead of being lost and reborn as a brand-new ID a moment later, which (tying back to Chapter 3) is precisely the kind of glitch that would otherwise cause one real violation to be reported as two separate events.

EXAMPLE — detection + ByteTrack tracking in one call

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Track objects across a video, each getting a persistent ID
results = model.track(
    source="rtsp://camera-1/stream",
    tracker="bytetrack.yaml",
    persist=True,
)
for r in results:
    for box in r.boxes:
        print(box.id, box.cls, box.xyxy)  # track ID stays stable across frames
```

With a stable track ID attached to every person, the system finally has what it needs to apply the temporal logic Chapter 3 called for: watch a specific person's PPE state over several seconds rather than a single frame, and only raise an alert once a violation has clearly persisted rather than flickered. The next chapter turns to the second half of the identity problem — not who someone is, but where they are, and how the system decides geometrically whether "where" counts as a violation.

Key Terms Introduced in This Chapter

Multi-object tracking (MOT) — Assigning a persistent identity to each detected object across a sequence of frames.

Track ID — The stable identifier attached to one specific tracked object over time.

Kalman filter — An algorithm that predicts an object's next position from its previous position and velocity, then corrects that prediction against real observations.

Data association — The problem of matching new detections to existing tracks each frame.

Hungarian algorithm — An efficient method for finding the best overall pairing between two sets (here, tracks and detections).

Occlusion — When an object is partly or fully hidden behind something else in the frame.

ByteTrack — A tracking algorithm that also uses low-confidence detections to keep tracks alive through occlusion, rather than discarding them.

CHAPTER 10

Geometry Rules: Matching PPE to People and Spotting Zone Intrusion

The system now knows, for each frame, where every person is and where every detected helmet and vest is, plus a stable ID linking each person across time. What's still missing is the logic connecting these pieces together: does *this* helmet belong to *that* person? And separately, has this person's position crossed into somewhere they shouldn't be? Both questions turn out to be solved with fairly simple geometry rather than more machine learning.

Matching PPE to a Person

Since the detector finds people, helmets, and vests as separate, independent boxes, the system needs a rule for associating them. A practical approach — simple enough to implement quickly, and robust enough to work well in practice — is purely positional: a detected helmet is considered "worn" by a person if the helmet box's center point falls within the upper region of that person's bounding box (roughly the top fifth, where a head would be); a detected vest is considered worn if its box overlaps substantially with the torso region of the person's box (roughly the middle third). If a person's box has no matching helmet or vest by these rules, that gap is itself the signal: the absence of a detection, in the right place, is what indicates a possible violation, rather than the presence of any "no-helmet" object needing to be detected directly.

Detecting Zone Intrusion

A restricted zone is defined by a safety officer as a polygon — a shape made of several connected points — drawn once on a still frame from a given camera and stored for reuse. To check whether a tracked person has entered that zone, the system needs to test whether a specific point (by convention, the bottom-center of the person's bounding box, which approximates where their feet are standing) falls inside that polygon. This is a classic computational-geometry task called a **point-in-polygon test**, and the most common way to solve it is **ray casting**: draw an imaginary ray from the point in question off to infinity in any fixed direction, and count how many times that ray crosses an edge of the polygon. An odd number of crossings means the point is inside; an even number means it's outside. It's a strange little fact of geometry, but a reliable one, and well-tested libraries implement it in a single function call.

EXAMPLE — point-in-polygon zone check (Shapely)

```
from shapely.geometry import Point, Polygon

# Zone drawn by a safety officer (normalized 0-1 coordinates)
restricted_zone = Polygon([(0.2, 0.4), (0.6, 0.4), (0.6, 0.9), (0.2, 0.9)])

feet_x, feet_y = 0.35, 0.6 # bottom-center of a tracked person's box
inside = restricted_zone.contains(Point(feet_x, feet_y))
print(inside) # True -> this person is inside the restricted zone
```

From a Single Frame to a Trustworthy Event

Both rules above, applied to one frame, would still suffer from the flicker problem raised in Chapter 3: a momentary misdetection could trigger a one-frame false alarm, and a person passing briefly through the edge of a zone would count the same as one who stopped and stayed. The fix is to think of each track's PPE and zone status as a small **state machine** rather than a single instantaneous check. Over a rolling window of the last several frames, the system keeps a **majority vote** of whether the person is currently compliant, which smooths over a single bad frame. A violation only escalates to a real event once it has persisted continuously for a minimum **dwelt time** — a couple of seconds is typical for zone intrusion — and, once raised, a **cooldown** period prevents the same track from re-triggering the same type of alert again immediately, which matters especially because a track's ID can occasionally switch (as discussed in Chapter 9) without the underlying violation actually starting over.

Key Terms Introduced in This Chapter

Point-in-polygon test — Determining whether a specific point lies inside a polygon-shaped region.

Ray casting — A method for solving point-in-polygon by counting how many times a ray from the point crosses the polygon's edges.

State machine — A model that tracks an entity's current status (e.g. compliant / in violation) and the rules for switching between statuses.

Dwell time — The minimum duration a condition must persist before it counts as a confirmed event.

Cooldown — A minimum waiting period before the same track can trigger the same alert type again.

CHAPTER 11

Live Video Streaming: RTSP and Why Live Is Harder Than Recorded

Every concept so far has been described in terms of "a frame," as though frames simply arrive, neatly, one after another, whenever the program asks for one. Working with a saved video file, that assumption mostly holds. Working with a live camera, it does not, and the gap between the two is a common source of frustration for anyone building this kind of system for the first time.

IP cameras on a site typically expose their live feed using a protocol called **RTSP** — the Real Time Streaming Protocol. Practically, this means a camera exposes a URL, such as `rtsp://192.168.1.50:554/stream1`, and any client that connects to it starts receiving a continuous stream of compressed video data. That compression matters: an uncompressed 1080p color frame is around six megabytes, and thirty of those every second would overwhelm almost any network, so cameras encode video using a **codec** — commonly H.264 — that shrinks it dramatically. The key detail worth knowing about how these codecs work is that they don't send every frame as a complete, independent image; instead, they periodically send a full **keyframe** (also called an I-frame), and then a run of much smaller frames that only describe what changed since the last one. This is efficient, but it means a client that joins the stream partway through has to wait for the next keyframe before it can decode anything at all — one small reason live streams have some inherent lag that a file played back from disk does not.

RTSP itself can run over two different transport methods: **TCP**, which guarantees delivery and correct ordering of every packet (at the cost of retransmitting anything lost, which can add delay), or **UDP**, which is faster and lower-latency but will simply drop a packet that gets lost on the network, potentially corrupting a frame. For a monitoring system where a single dropped frame is a non-event but a frozen or crashed connection is a real problem, TCP's reliability is normally the safer default, and most computer-vision libraries let you force it explicitly.

EXAMPLE — connecting to an RTSP stream over TCP (OpenCV)

```
import os
import cv2

# Force RTSP to use TCP transport for a more stable connection
os.environ["OPENCV_FFMPEG_CAPTURE_OPTIONS"] = "rtsp_transport;tcp"

cap = cv2.VideoCapture("rtsp://192.168.1.50:554/stream1")
ok, frame = cap.read()
```

There's a second, equally important difference between live and recorded video: speed mismatch. A saved file can be processed at whatever pace the program manages, pausing

and resuming freely — nothing is lost by taking an extra tenth of a second on a slow frame. A live camera keeps sending frames in real time regardless of whether the program is ready for them, and if processing falls behind even slightly, a naive program that tries to read and process every single frame in strict order will fall further and further behind, building up a growing backlog of stale frames — meaning the "live" view the system is reacting to might actually be many seconds in the past. The standard fix is architectural: run frame reading on its own dedicated background **thread**, continuously pulling the newest frame into a small buffer (a **queue**) that holds only the latest one or two frames, deliberately discarding older ones the processing side hasn't gotten to yet. The processing loop then always grabs whatever is freshest in that queue. This producer-consumer pattern, running reading and processing as two loosely coupled loops rather than one tightly locked sequence, is what lets the system stay caught up with several live cameras at once — and it's also why Python's **threads** are a reasonable fit here even though Python has a well-known limitation called the **GIL** (Global Interpreter Lock, which prevents true parallel execution of Python code across threads): most of the actual waiting in this design happens on network I/O, which releases the GIL anyway, so threads work well for reading multiple streams even though they wouldn't help much for CPU-heavy number crunching.

Key Terms Introduced in This Chapter

RTSP — Real Time Streaming Protocol — the standard way IP cameras expose a live video feed as a URL.

Codec — A method for compressing and decompressing video (e.g. H.264).

Keyframe (I-frame) — A complete, independently decodable frame; frames between keyframes only encode what changed.

TCP vs. UDP — Two network transport methods: TCP guarantees delivery and order; UDP is faster but can silently drop data.

Thread — A separate line of execution within a program, letting one task (e.g. reading frames) run alongside another (e.g. processing them).

Queue — A buffer that holds items (here, frames) waiting to be processed.

GIL (Global Interpreter Lock) — A Python-specific restriction that prevents two threads from executing Python bytecode at the exact same instant.

CHAPTER 12

Making Models Fast: ONNX, TensorRT and Quantization

A YOLO model trained and tested in Chapter 8 runs perfectly well for experimentation, but the framework it was trained in — usually PyTorch — is built for flexibility (easy to modify, easy to debug, easy to retrain) rather than raw execution speed. When the goal shifts from "does this model work" to "can this model process eight live camera streams at once on one modest machine," a second, distinct engineering problem appears: making a working model run fast, without meaningfully hurting its accuracy. This chapter is about the tools built specifically for that job.

The first tool is **ONNX** (Open Neural Network Exchange), a shared file format for trained models that isn't tied to any one framework. Exporting a PyTorch model to ONNX captures its full structure and learned weights in a standardized way that many other tools — including the specialized inference engines this chapter is really about — know how to read. Think of it as similar to exporting a document to PDF: the original editable file might be locked to one piece of software, but the exported version can be opened and used almost anywhere.

EXAMPLE — exporting a trained YOLO model to ONNX

```
from ultralytics import YOLO

model = YOLO("best.pt")          # your fine-tuned PPE model
model.export(format="onnx")       # writes best.onnx
```

The second, more aggressive tool is **TensorRT**, a library from NVIDIA that takes a model (often via its ONNX export) and compiles it into a highly optimized execution plan specifically tuned for the exact GPU it will run on. It does this through several tricks that are worth having a mental picture of, even without their full engineering detail: **layer fusion**, which merges several small mathematical operations that would otherwise run as separate steps into one combined operation, cutting down on overhead; and automatic selection of the fastest available low-level GPU routine for each operation, out of several the hardware supports. The output of this process is called a TensorRT **engine** — and a crucial practical detail is that this engine is compiled specifically for the GPU (and TensorRT version) it was built on, so it isn't portable the way the original ONNX file was; you build it once, on the machine you intend to deploy on.

The third tool, often used alongside the first two, is **quantization** — reducing the numerical precision the model computes with. A model is normally trained using 32-bit floating-point numbers (**FP32**), which are precise but relatively slow and memory-hungry to compute with. Converting the model to use 16-bit floats (**FP16**) roughly halves memory use and often nearly doubles throughput on modern GPUs, typically with an accuracy loss

too small to notice. Going further still, to 8-bit integers (**INT8**), can roughly double speed again, but the accuracy cost becomes noticeable unless the conversion is guided by a small **calibration dataset** — a representative sample of real images the quantization process uses to figure out how to map the wider range of FP32 values down onto INT8's much coarser scale without losing the numeric distinctions that actually matter for the model's predictions.

All three tools together are why Part IV's Phase 4 treats this as a benchmarking exercise rather than a single fixed recipe: FP32, FP16 and INT8 each sit at a different point on the same trade-off between speed and accuracy, and the right choice depends on the specific hardware available and how much accuracy the project's mAP@50 target can afford to give up in exchange for supporting more camera streams at once.

Key Terms Introduced in This Chapter

Inference — Running a trained model to get predictions on new data (as opposed to training it).

ONNX — A shared, framework-independent file format for trained models.

TensorRT — An NVIDIA library that compiles a model into a highly optimized engine for a specific GPU.

Layer fusion — Merging multiple small computational steps into one combined, more efficient operation.

Quantization — Reducing the numeric precision a model computes with, to trade a small amount of accuracy for speed and lower memory use.

FP32 / FP16 / INT8 — Progressively lower-precision number formats a model can be converted to run in.

Calibration dataset — A small, representative set of real images used to guide accurate INT8 quantization.

CHAPTER 13

The Backend: APIs, Databases, Queues and Alerts

Everything covered so far — detection, tracking, zone rules, fast inference — lives inside what you could call the vision engine: the part of the system that watches video and decides "a violation just happened." That decision is worthless, though, until it reaches a human being and gets recorded somewhere reviewable. That's the job of the **backend**: the collection of ordinary software-engineering components that store violations, serve them to a dashboard, and push alerts out to a phone or inbox.

APIs and FastAPI

A **REST API** is simply an agreed-upon way for one piece of software to ask another to do something over a network, using plain HTTP requests — the same technology a web browser uses to load a page. Instead of returning a web page, though, an API endpoint returns structured data (usually as **JSON**) and often performs an action, like "save this new violation record." This project uses **FastAPI**, a Python framework for building these endpoints with very little boilerplate code, which is what the dashboard (and, later, external tools) will actually talk to.

EXAMPLE — a minimal FastAPI endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Violation(BaseModel):
    camera_id: str
    violation_type: str
    confidence: float

@app.post("/violations")
def create_violation(v: Violation):
    # save v to the database here
    return {"status": "recorded"}
```

Databases

A **database** is where information is stored so it survives after the program that created it stops running, and so many different parts of the system can read and write it safely at once. This project uses **PostgreSQL**, a relational database that organizes information into **tables** — think of a table as a spreadsheet with a fixed set of columns, where each **row** is one record. A violations table, for instance, might have columns for camera ID, violation type, track ID, timestamp, confidence, a path to the saved snapshot image, and a status

(new, confirmed, or false alarm) that the safety officer sets from the dashboard.

Queues, Alerts and Why They're Decoupled

The last piece is alert delivery — sending a message to Telegram, or an email, when a violation is confirmed. This sounds like it should be the simplest part of the system, but there's a subtle trap: sending a message over the internet to Telegram's or an email provider's servers can take anywhere from a few dozen milliseconds to several seconds, especially if that service is briefly slow or unreachable. If the vision engine called that alerting code directly, in the same loop that's also trying to process the next video frame, a single slow notification would stall detection on every camera the system is watching. The fix, once again, is the producer-consumer pattern from Chapter 11: the vision engine, on confirming a violation, simply drops a small message onto a **queue** and immediately moves on to the next frame; a separate worker process reads from that queue at its own pace, handles the actual network call to Telegram's **Bot API** or an email server via **SMTP**, and retries automatically if a send fails. This decoupling — keeping the time-critical vision loop completely isolated from the unpredictable timing of external network calls — is a small piece of design that matters far more in practice than it might seem on paper, because it's what keeps a temporary hiccup in your email provider from quietly degrading detection on every camera in the building.

With every concept from Part II now on the table — images as arrays, detection with YOLO, identity through ByteTrack, geometry rules for PPE and zones, the realities of live streaming, model optimization, and the backend that ties it together — the next part steps back and shows how all of these pieces fit into one running system.

Key Terms Introduced in This Chapter

REST API / endpoint — A standardized way for software to request an action or data from another program over HTTP; an endpoint is one specific URL that performs one action.

JSON — A lightweight, structured text format commonly used to send data between programs.

Database / table / row — Persistent storage for structured data; a table is like a spreadsheet, a row is one record in it.

Message queue — A buffer that decouples a fast, time-critical process (like video inference) from a slower one (like sending a notification).

SMTP — The standard protocol used to send email.

PART III

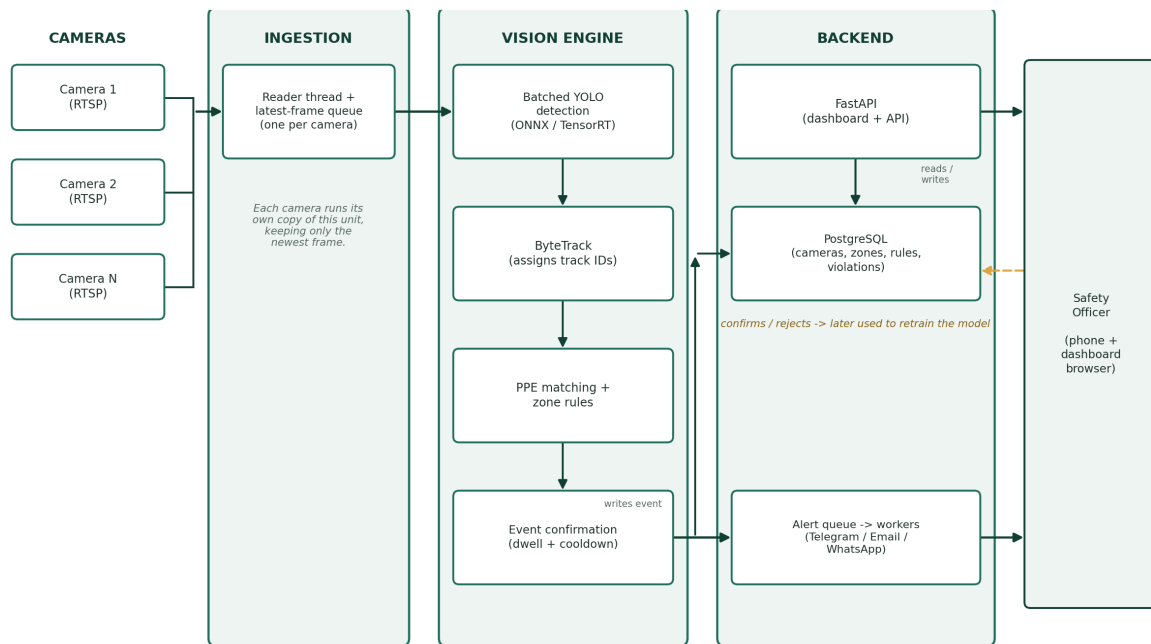
System Architecture

One diagram, one full walkthrough of a single violation, and the reasoning behind the key design decisions.

CHAPTER 14

The Big Picture: End-to-End Architecture

Part II covered nine ideas in isolation — pixels, detection, tracking, geometry, streaming, optimization, and backend systems. This chapter puts them back together into the one diagram that matters most: how data actually moves through the finished system, from a camera lens to a safety officer's phone.



Cameras and Ingestion

Each camera exposes an RTSP feed (Chapter 11). Rather than one shared reader, every camera gets its own dedicated reader thread and its own small latest-frame queue — a deliberate design choice covered in more depth in Chapter 16, but the short version is that a slow or stalled camera should never be allowed to hold up any of the others.

The Vision Engine

This is where Chapters 6 through 10 live in code. Frames pulled from the ingestion queues are batched together and run through YOLO, optimized via ONNX/TensorRT (Chapter 12) so this step stays fast enough for several cameras at once. The raw detections then pass to ByteTrack (Chapter 9), which turns them into stable track IDs, and those tracked people are checked against the PPE-matching and zone rules from Chapter 10. The final step, event confirmation, is the dwell-time-and-cooldown state machine that turns a persisting rule violation into exactly one trustworthy event — the mechanism that solves the flicker

problem raised all the way back in Chapter 3.

The Backend

A confirmed event forks in two directions at once, and this fork is intentional: one copy is written straight to PostgreSQL as a permanent record, and a second, much smaller message is dropped onto the alert queue. From there, a separate pool of alert workers — decoupled from the vision engine exactly as Chapter 13 described — handles the actual Telegram, email, or WhatsApp delivery, retrying on failure without ever touching the time-critical detection loop. FastAPI sits in front of PostgreSQL, serving the dashboard the safety officer actually uses.

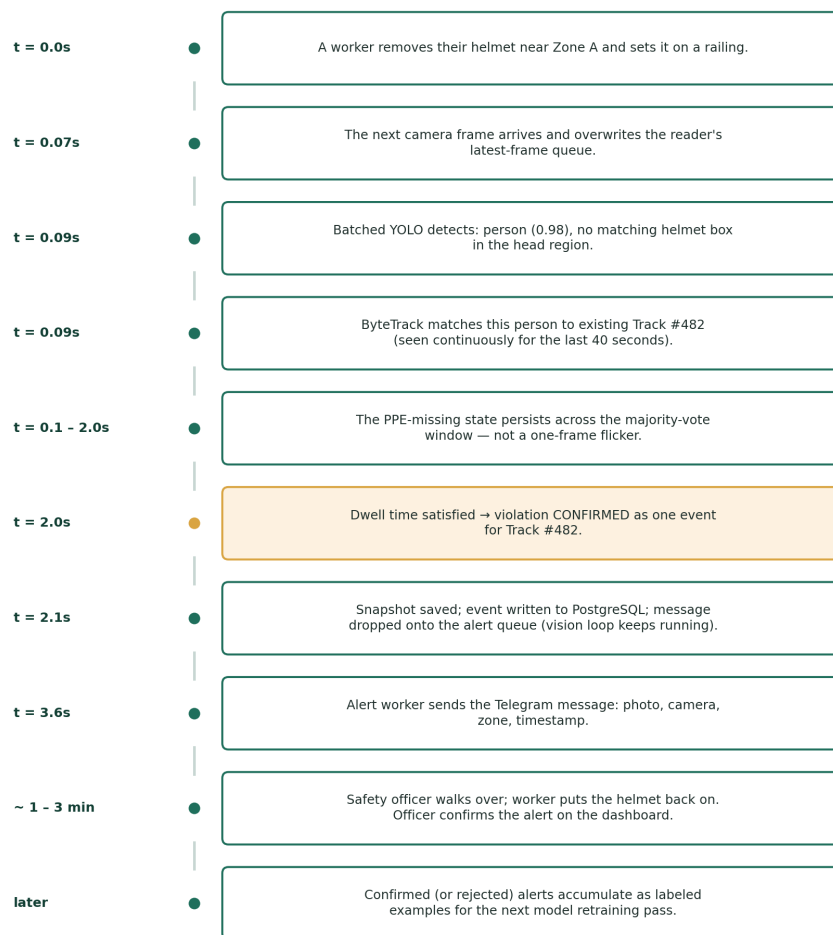
The Feedback Loop

One connection in the diagram is easy to overlook but matters as much as any other: the dashed line from the safety officer back into PostgreSQL. Every time an officer marks an alert "confirmed" or "false alarm," that label becomes part of the permanent record too — and, as Chapter 4 described, this is exactly the raw material Phase 6 later uses to retrain the model on its own real mistakes rather than only on the original training set. A system with no path for this feedback to get back into the model can only ever be as good as its first training run; this one is built to improve.

CHAPTER 15

Life of a Violation: A Step-by-Step Walkthrough

Diagrams of components are useful, but they can obscure something important: none of this happens in stages the way an org chart might suggest. It happens as one continuous, fast sequence, in response to one small real-world moment. This chapter traces a single violation, start to finish, through every system covered so far, with approximate timings to make the pacing concrete.



Notice how much happens in the first tenth of a second — frame capture, detection, and tracking are all essentially instantaneous compared to human perception — while the two-second dwell time before confirmation (Chapter 10) is, by comparison, deliberately unhurried. That asymmetry is intentional. The expensive-to-get-wrong decision is whether to alert a human being at all, so the system takes its time there; the cheap, repeatable

decisions underneath it — one more detection, one more tracking update — can and should run as fast as the hardware allows.

Notice, too, where the timeline branches into two independent speeds. Up through $t = 2.1$ seconds, everything happens inside the tight, latency-sensitive vision loop. The moment the event is queued, the vision engine's job for this particular violation is done — it has already moved on to the next frame from this camera and every other one. Everything from $t = 3.6$ seconds onward (the Telegram delivery, the officer's walk across the site, the eventual dashboard confirmation) happens on entirely different, much slower timescales, in different processes, without either side blocking the other. This split — fast, tight loop on one side; slow, human-paced follow-up on the other, joined only by a queue and a database row — is arguably the single most important structural idea in the whole system, and it's worth keeping in mind as the reason several of Part IV's implementation choices are made the way they are.

CHAPTER 16

Design Decisions and Trade-offs

Every non-trivial system embeds dozens of small decisions that could reasonably have gone another way. Making those decisions explicit — and explaining what each one trades away — is often more educational than describing what was built, because it's the part a finished system's code doesn't show you. This chapter walks through the project's most consequential trade-offs.

Why a "Latest-Frame-Only" Queue, Not a Full Buffer?

A frame queue could be built to hold everything the camera sends, guaranteeing that the vision engine eventually processes every single frame. This project deliberately does the opposite (Chapter 11): the queue holds only the one or two newest frames and silently discards anything older once processing falls behind. The trade-off is completeness for freshness. For a monitoring system, being three seconds behind on a live camera is far more dangerous than skipping a frame outright — a stale "live" view could mean reacting to a violation that has already ended and missing the one happening right now. A system that must never miss a single frame (forensic evidence capture, say) would make the opposite choice.

Why TCP, Not UDP, for RTSP?

Chapter 11 mentioned this briefly: TCP guarantees delivery at the cost of occasional retransmission delay, while UDP is faster but drops data silently on a bad network. This project defaults to TCP because a dropped or corrupted frame here isn't a catastrophe (the next frame arrives a fraction of a second later, and detection runs again), but a connection that silently degrades without the program noticing is much harder to detect and debug. Predictability wins over raw speed at the network layer, since the speed that actually matters for this project is measured after decoding, in the vision engine, not on the wire.

Why ByteTrack, Not a Heavier Re-Identification Tracker?

Some trackers go further than ByteTrack by learning a visual "fingerprint" for each person (called re-identification, or re-ID) so they can be recognized again even after leaving the frame entirely and coming back minutes later, or reappearing on a different camera. That capability costs meaningfully more compute per frame — exactly the resource this project is already trying to conserve across several simultaneous streams (Chapter 12). ByteTrack's more modest goal, keeping an ID stable through a brief occlusion within one continuous camera view, matches what this project actually needs: a violation only has to stay attributed to one person for the length of that one violation, not be re-recognized across the whole shift.

Why Match PPE by Geometry, Not by Training a Fused Detector?

An alternative design would train YOLO directly on combined classes like "person-wearing-helmet" and "person-not-wearing-helmet," skipping the separate geometric matching step in Chapter 10 entirely. This can work, but it multiplies the labeling effort (every combination of PPE state needs its own labeled examples) and produces a model that has to be retrained from a meaningful chunk of scratch every time a new PPE rule is added — a new class means new data, not a config change. Keeping detection (find people, helmets, vests) and reasoning (whose helmet is whose, is this a violation) as two separate, simpler steps costs a small amount of geometric approximation, but buys flexibility: adding a new rule, such as requiring safety glasses in one specific zone, becomes a change to the rules engine rather than a full retraining cycle.

Why Decouple Alerting Through a Queue?

Chapter 13 already made the core argument: a slow or failing external API call must never be allowed to stall video processing. It's worth adding the trade-off explicitly — this decoupling adds real complexity (a queue is one more moving part that can, in principle, itself get stuck or lose messages, and needs its own monitoring). The project accepts that added complexity because the alternative failure mode — a hung network call silently freezing detection on every camera — is far worse and far harder to notice while it's happening.

Why PostgreSQL, Not a NoSQL Store?

The project's data — cameras, the zones defined on them, the rules attached to those zones, and the violations that reference all three — is inherently relational: a violation always belongs to exactly one camera and (optionally) one zone, and a dashboard query like "show me every unresolved violation in Zone A this week" is naturally expressed as a join across those relationships. A relational database with proper foreign keys enforces that these references stay valid automatically; a document-oriented NoSQL store would leave that consistency to be maintained by hand in application code. For data shaped like this, the trade-off favors a relational database.

Why Target FP16 by Default, Rather Than Always INT8?

Chapter 12 introduced the FP32 → FP16 → INT8 spectrum. This project treats FP16 as the default target and INT8 as an optional stretch goal to benchmark, not the other way around. FP16 reliably delivers a large speed gain with accuracy loss too small to matter for almost any use case; INT8's extra speed gain is real, but its accuracy cost varies significantly depending on how representative the calibration dataset is, and a poorly calibrated INT8 model can silently under-detect exactly the rare, hard cases (a helmet at an unusual angle, an unusually dim corner of a site) that matter most for a safety system.

The extra speed is worth chasing, but only after confirming the accuracy cost with real measurement, per Phase 4's benchmarking step.

How This Would Change at Larger Scale

It's worth being honest that this architecture is scoped for a single site with a handful to a few dozen cameras running on one to a few machines — appropriate for a learning project or a single-facility deployment, but not for hundreds of cameras across many sites. At that scale, several of today's simplifications would need to be revisited: the simple in-process alert queue would likely become a dedicated message broker (such as Kafka or RabbitMQ) shared across many machines; the single vision-engine process would become a fleet of GPU workers behind a load balancer, with cameras assigned dynamically rather than one-process-per-site; and PostgreSQL would likely need read replicas or partitioning to keep dashboard queries fast as the violations table grew into the hundreds of millions of rows. None of that complexity is needed to build and demonstrate the system this book describes, which is exactly why it isn't part of the design — but knowing where the current design's ceiling is, and why, is itself a useful thing to be able to explain.

PART IV

Building It: The Phase-Wise Guide

Seven phases, from an empty repository to a benchmarked, working system — plus a week-by-week study plan for learning everything above as you go.

CHAPTER 17

Phase 0 — Setup and Foundations

Week 1, first two days

Goal

Get a repository, a development environment, and a source of test video in place before writing any vision code. Rushing past this phase is the single most common cause of wasted time later — a CUDA/driver mismatch discovered in Phase 4 is far more expensive to fix than one caught here.

Concepts You'll Apply

- Chapter 12's mention of matched CUDA / driver / TensorRT versions
- Chapter 11's RTSP basics, used here to build a fake camera to develop against

Step by Step

- Create the repository with a clear folder layout (e.g. *ingestion/*, *vision/*, *rules/*, *backend/*, *dashboard/*) and a Python virtual environment.
- Pin CUDA, GPU driver and (if using a GPU) TensorRT versions together now, and write them down in the README — mismatches between these three are the most common source of mysterious failures later.
- Set up a fake CCTV network so you can develop without needing real site access: run MediaMTX as a local RTSP server, and use ffmpeg to loop a handful of sample videos into it as simulated cameras.
- Collect 5–10 short (1–3 minute) test clips showing workers with and without helmets and vests, ideally in different lighting and camera angles.
- Decide the class scheme now: either train explicit violation classes (e.g. "NO-Hardhat") or detect person / helmet / vest separately and match them geometrically (Chapter 10's approach, and the one this book uses, since it's more robust to noisy labels and easier to extend).

EXAMPLE — simulating an RTSP camera with ffmpeg + MediaMTX

```
# Loop a sample video into a fake RTSP camera for development
ffmpeg -re -stream_loop -1 -i sample_site_clip.mp4 \
  -c copy -f rtsp rtsp://localhost:8554/camera1
```

Common Pitfalls

- Skipping the fake-camera setup and trying to test against a real, distant site camera from day one — slow and unreliable for daily development.
- Not pinning versions, then hitting a confusing TensorRT/driver mismatch weeks later with no memory of what was installed when.

Done when: a live RTSP test stream from your fake camera plays back correctly in OpenCV on your development machine.

AS BUILT · Verified on the Mac (Apple M4), 21 September 2026

What was built

- A Python project (*ppe-monitor*) with the folder layout above, every version pinned (PyTorch 2.14, OpenCV 4.14, NumPy 2.3.5, Ultralytics 8.4, MediaMTX 1.21), an environment report written by a script, and a one-command setup script for the Mac.
- The fake CCTV network: MediaMTX as the RTSP server and one ffmpeg process per camera, looping a clip and encoding it on the Mac's hardware video encoder. Three synthetic cartoon clips with scripted moments — a helmet taken off, a worker with no vest, a person walking into a restricted zone.
- A stream viewer and checker that measures frame rate, stalls and grey frames; a one-shot acceptance check that starts and stops everything itself; and an automated test suite, including an end-to-end test from clip to ffmpeg to MediaMTX to OpenCV.

What changed from the plan, and why

- **No NVIDIA GPU.** The development machine is a MacBook Air (Apple M4). PyTorch runs on its GPU through Apple's Metal ("MPS") instead of CUDA, and TensorRT — which only exists for NVIDIA — is replaced in Phase 4 by Core ML and ONNX. "Pin CUDA, driver and TensorRT" became "pin PyTorch, OpenCV, NumPy and Ultralytics, and write them into a report".
- **No real clips yet.** Synthetic clips tested the plumbing; real footage arrived later (Chapter 23's build notes) and changed the picture considerably.

Measured results

Check	Result
Acceptance check: 3 simulated cameras	PASS — 15.0 frames/s each, longest gap 0.09 s
Apple GPU check (a YOLO-sized network, 640×640)	6.7× faster than the CPU in FP32, 9.0× in FP16
Opening a stream (low-latency settings)	0.8 s and 4 frames behind live (defaults: 2.8 s, 31 frames)

Lessons from the build

- **Settings can leak.** OpenCV reads FFmpeg's low-latency options from a process-wide variable. Left set, every video *file* opened afterwards silently lost half its frames — which would have quietly corrupted every later evaluation. They are now set only while a stream opens, and a test guards it.
 - **Loops can freeze.** Each fake camera froze for about a second whenever its clip looped: ffmpeg decoded on one thread per CPU core, and that pipeline empties at every loop (1.1 s on the 10-core M4). One decoding thread fixed it, and the acceptance check now reads past the loop point.
-

CHAPTER 18

Phase 1 — Data and Baseline Detector

Week 1

Goal

Get a working, if imperfect, PPE detector trained and measured, so every later phase has something real to build on top of rather than a hypothetical model.

Concepts You'll Apply

- Chapter 6 — bounding boxes, IoU, precision/recall, mAP@50
- Chapter 7 — how a CNN learns from labeled examples
- Chapter 8 — pretrained weights, transfer learning and fine-tuning with YOLO

Step by Step

- Pull one or two PPE / construction-safety datasets from Roboflow Universe. Check each one's license, class names, and label quality before committing to it, and map differing class names onto one consistent list.
- Hold out a test set from scenes the model will never train on — ideally your own recorded clips from Phase 0, not just a random split of the training data. Random splits of similar public datasets often contain near-duplicate frames across train and test, which quietly inflates measured mAP without the model actually being better.
- Fine-tune a small pretrained YOLO variant (Chapter 8) on the combined dataset, and record per-class mAP@50 on your held-out test set, not just the training set's numbers.
- Manually review a sample of the model's mistakes. Look specifically for the patterns Chapter 3 predicted: distant workers, partial occlusion, night or backlit scenes, caps mistaken for helmets, bright clothing mistaken for vests.

EXAMPLE — fine-tuning and validating a YOLO model

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
model.train(data="ppe_dataset.yaml", epochs=50, imgsz=640)
metrics = model.val()          # runs on the held-out test set
print(metrics.box.map50)      # mAP@50
```

Common Pitfalls

- Reporting mAP@50 from the training split instead of a genuinely held-out test set — this is the single most common way early results look better than they really are.
- Combining datasets with inconsistent class definitions (one dataset's "vest" including reflective jackets, another's not) without checking first.

Done when: you have a baseline model, a per-class mAP@50 number from a genuinely held-out test set, and a written list of specific failure cases.

AS BUILT · Built and reviewed, 23 September 2026

What was built

- Two public datasets merged into one with three classes — person, helmet, vest: Construction-PPE (Ultralytics, AGPL-3.0) and Construction Safety from Roboflow 100 (CC BY 4.0), each downloaded and checked against a pinned checksum. Result: 2,170 training, 237 validation and 200 test images.
- A leak check: every image is fingerprinted, and groups of near-identical photos that span splits are moved into training, so no test image has a twin the model trained on.
- YOLO26n fine-tuned on the Mac's GPU (54 epochs, stopped early), a per-class evaluation, a gallery of the model's mistakes with the reason for each, and a model card.

What changed from the plan, and why

- **YOLO26n instead of YOLOv8n** — the newer small model of the same family, following Chapter 8's advice to use what the library currently recommends.
- **The test set is the datasets' own test split** (duplicates removed), not your own clips, because there were none yet. It measures these datasets honestly, but contains almost no small, distant people — the gap the first real clips later exposed.
- **"Vest" means any hi-vis garment** (vest, jacket, coverall), decided after looking at how each dataset used the label; harnesses and life jackets are not vests.

Measured results

Class (test split)	AP@50	Precision	Recall
person	0.838	0.826	0.848
helmet	0.921	0.880	0.857
vest	0.849	0.852	0.789
all (mAP@50)	0.869		

Lessons from the build

- **Many "false alarms" were real people the labellers skipped.** Label gaps look exactly like model mistakes until you look at the pictures — which is why the mistake gallery exists.
- **The failure list predicted the future.** Crowds, small people, and white and blue helmets were on it on day two; the first real workshop footage failed in exactly those ways.
- An accidental second training run, in parallel, produced an identical model: with the random seed fixed, training is reproducible.

CHAPTER 19

Phase 2 — Tracking and PPE Logic

Week 2

Goal

Turn frame-by-frame detections into stable, tracked people, and turn raw detections of helmets and vests into a real judgment of who is and isn't compliant, smoothed over time.

Concepts You'll Apply

- Chapter 9 — ByteTrack, track IDs, occlusion handling
- Chapter 10 — geometric PPE matching, majority voting, dwell time and cooldown

Step by Step

- Enable ByteTrack explicitly in Ultralytics' tracking mode so every detected person receives a persistent track ID across frames.
- Implement the PPE-matching rule from Chapter 10: a helmet counts as worn if its box center falls in the upper region of the person's box; a vest counts as worn if its box substantially overlaps the torso region.

- Add temporal smoothing: keep a rolling majority vote of each track's compliance state over the last several frames, so one bad frame doesn't flip the verdict.
- Add the confirmation state machine: a violation only escalates to a real event once it's persisted for a minimum dwell time (2–3 seconds is a reasonable starting point), and a cooldown prevents the same track from re-triggering the same alert type immediately afterward.

EXAMPLE — streaming detections with persistent track IDs

```
from ultralytics import YOLO

model = YOLO("best.pt")
for r in model.track(source="rtsp://localhost:8554/camera1",
                    tracker="bytetrack.yaml", persist=True, stream=True):
    for box in r.bboxes:
        track_id = int(box.id) if box.id is not None else None
        # feed (track_id, class, xxyy, confidence) into your PPE-matching logic
```

Common Pitfalls

- Testing PPE-matching logic only on clean, front-facing footage — verify it against your Phase 0 clips with awkward angles and occlusion too.
- Setting dwell time too short (re-introduces flicker) or too long (real violations take too long to alert on) without testing both directions.

Done when: on offline test videos, each real, sustained violation produces exactly one confirmed event, and momentary misdetections produce none.

AS BUILT · Built and reviewed, 24 September 2026

What was built

- The chain: detector → person tracker (ByteTrack) → PPE rule per person → event engine → event with a snapshot, for a video file or a live camera.
- A PPE rule with three answers — **worn**, **missing** and **unknown**. Unknown (the person is too small, cut off by the frame edge, or bent over with the head out of sight) never raises an alarm.
- Body keypoints from a second small model (YOLO26n-pose): where the head really is, and whether the person is upright, bending, crouching or lying down.
- The event engine: a 1-second majority vote, a 3-second dwell, a cooldown, and incidents that follow the person — so one violation gives one alert even when the tracker hands that person a new ID.
- 86 short test clips made from the test photos, still and with everyone moving at walking pace, to score whole events rather than single frames.

What changed from the plan, and why

- **Regions from measurement.** The head region runs from 10% of the person's height above the box to 35% into it; a vest's centre must sit between 15% and 85%. These came from where 5,900 labelled helmets and vests actually sit. Chapter 10's tighter rules of thumb judged 8.8% of helmet wearers and 13.6% of vest wearers "missing".
- **Keypoints were added** when the review asked for walking, bending and crouching workers: helmets found on people bending over rose from 92% to 98%, and on people lying down from 0% to 55%.
- **The tracker needed repairs for moving people:** a new track that appears where a lost person would be by now takes over that person's ID.
- **A 3-second dwell rather than 2:** a simulation showed 3 s removes every single-frame and half-second burst of detector errors.

Measured results

On the 86 test clips	Still	Moving, 15 fps	Moving, 5 fps
No helmet: caught exactly once	25 / 28	42 / 47	38 / 47
No vest: caught exactly once	35 / 45	55 / 68	48 / 68
People with two alerts	0	0	5
False alerts (helmet / vest)	4 / 6	5 / 6	5 / 3

The whole chain, with keypoints, takes 27–29 ms per frame on the Mac.

Lessons from the build

- **Frame rate matters for groups.** At 5 frames a second, people walking close together swap IDs and a few get two alerts — the reason Phase 4 keeps every camera at 10.
- **The detector was the easy part.** Turning its per-frame answers into exactly one alert per real violation took most of the phase.

CHAPTER 20

Phase 3 — Restricted Zones and Rules Engine

Weeks 2–3

Goal

Let a safety officer define restricted zones per camera, and build a rules engine general enough to express more than just "no entry" — severity, active hours, and per-zone dwell times included.

Concepts You'll Apply

- Chapter 10 — point-in-polygon zone checks and ray casting

Step by Step

- Build a small zone-drawing tool: click points on a still frame from a given camera to define a polygon, then store it as normalized (0-1) coordinates so it survives resolution changes. A quick OpenCV script is enough for a first version; a browser-based canvas is a natural later upgrade.
- Implement the intrusion check from Chapter 10: test whether a tracked person's bottom-center point (their approximate foot position) falls inside the stored polygon, requiring a short dwell time before counting it as a real intrusion.
- Design the rules engine to be config-driven rather than hard-coded: each rule specifies its type, the camera and zone it applies to, a severity level, a dwell time, a cooldown, and optionally active hours (a loading-dock zone might only be restricted during active crane operations, for instance).
- Unit-test the rules engine directly with synthetic, hand-written tracks — no video required for this step, which makes edge cases much faster to check.

Common Pitfalls

- Storing zone coordinates in raw pixels instead of normalized coordinates — a camera resolution change silently corrupts every stored zone.
- Hard-coding rule logic per zone instead of driving it from configuration, making every new rule a code change instead of a data change.

Done when: both the PPE rules from Phase 2 and the zone rules here emit the same structured event shape — camera, type, track ID, time, box, and snapshot frame — so the rest of the system can treat them identically.

AS BUILT · Built, checked on the Mac and reviewed, 24 September 2026

What was built

- A zone drawing tool: click the corners on a frame from the camera. Zones are stored as 0-1 fractions of the frame, checked when loaded, and saved with the frame they were drawn on.
- The zone test on the point a person stands on: their ankle keypoints when visible, otherwise the bottom of their box. Feet out of view means "can't tell", never an alert.
- A rules engine driven by two configuration files: rule type (no helmet, no vest, zone intrusion), cameras, zone, severity, dwell, cooldown and active hours — several ranges, overnight, chosen days, a time zone.

- One event shape for every rule, written to a log with its snapshot.

What changed from the plan, and why

- **Ankles before the box.** A box is wrong when an arm is outstretched or two people merge into one box; ankles aren't.
- **Our own point-in-polygon test** (about ten lines of ray casting) instead of a geometry library: one dependency fewer, and it handles any zone shape.

Measured results

Check (on the Mac)	Result
Walked into a zone and stayed: exactly one zone event	27 of 37 people
People with two zone events	0
False zone events (88 people never inside)	0
Feet crossing the line → alert	2.6 s median (the rule's dwell is 2 s)
Simulated cameras	keep-out zone fired once; the pit zone was on at noon and off after 19:00, as set

Lessons from the build

- **Every miss was upstream.** Small people the detector never found, groups merged into one box, feet at the frame edge — the zone logic missed nothing it could see.
- **A placement rule for real sites:** a camera guarding a zone must see the whole zone, and people's feet in it.

CHAPTER 21

Phase 4 — Multi-Stream Ingestion and Optimization

Weeks 3–4

Goal

Scale from one test stream to several concurrent live cameras, and make the detector fast enough that doing so doesn't require a data-center GPU.

Concepts You'll Apply

- Chapter 11 — RTSP, TCP transport, threads, and the latest-frame queue pattern
- Chapter 12 — ONNX export, TensorRT engines, FP16/INT8 quantization

Step by Step

- Give each camera its own reader thread and a small queue that always drops stale frames, exactly as Chapter 11 described. Add automatic reconnection with backoff for cameras that drop out.
- Force RTSP to use TCP transport for stability, and process each stream at a fixed, modest rate (5–10 FPS is often enough) rather than whatever rate the camera happens to push.
- Batch frames from multiple streams into single inference calls where possible, rather than running the model once per camera sequentially.
- Export the fine-tuned model to ONNX, then build a TensorRT engine from it. Keep the plain ONNX / CPU path working as a fallback for machines without a compatible GPU.
- Benchmark FP32 vs. FP16 vs. INT8 explicitly: measure latency, FPS, and GPU/CPU utilization, and re-measure mAP@50 at each precision level, with 1, 2, 4 and 8 simulated streams running at once.

Common Pitfalls

- Benchmarking only with a single stream — throughput problems often only appear once several cameras compete for the same GPU.
- Adopting INT8 purely because it's fastest, without checking the mAP@50 drop it caused on your specific dataset.

Done when: you have a throughput-vs-accuracy table across precision levels and stream counts, and the pipeline survives a camera disconnecting and reconnecting without crashing or stalling the others.

AS BUILT · Done, 25 September 2026: eight cameras at 10 frames a second on one Mac

What was built

- One reader thread per camera that keeps only the newest frame, reconnects after 0.5, 1, 2, 4 and 8 s and then every 10 s, and always uses TCP.
- One shared clock: ten times a second, the newest frame of every camera goes through the detector in one batch, then through the keypoint model; each camera keeps its own tracker, rules and events.
- The models exported four ways — Core ML FP16 and 8-bit (on the Neural Engine), ONNX FP32 and INT8 (on the CPU) — a benchmark of speed against accuracy with 1, 2, 4 and 8 cameras, and a check that unplugs a camera and plugs it back in.
- The keypoint model on 1 frame in 2 of each camera. In between, each person's last keypoints are moved onto their new box: posture hardly changes in a tenth of a second.

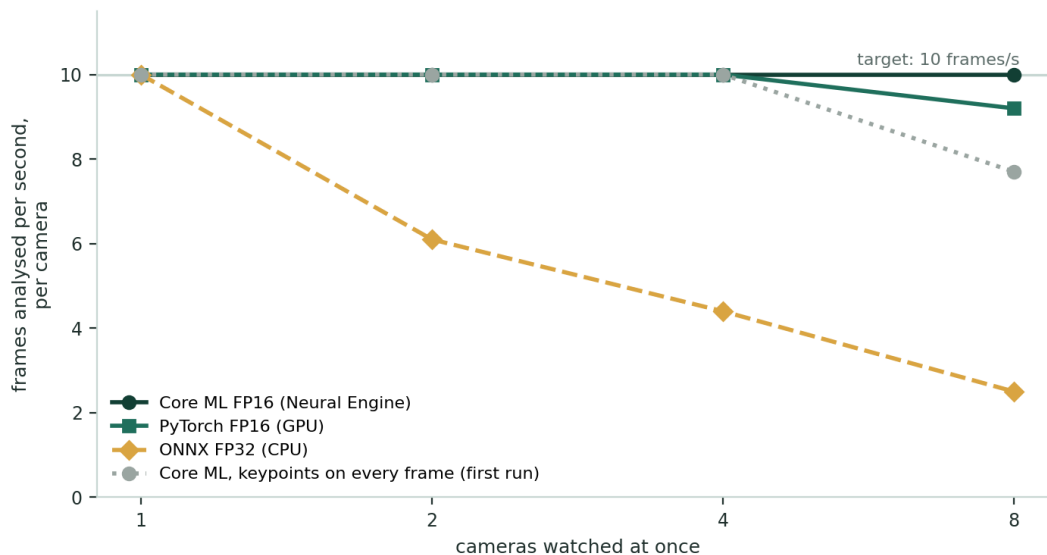
What changed from the plan, and why

- **Core ML and ONNX instead of TensorRT** (no NVIDIA GPU). Core ML on the Neural Engine is the fastest here, and the default.
- **One clock for all cameras.** The first version gave each camera its own timer; the timers drifted apart and no two frames were ever batched together. A test caught it.
- **Reconnection waits capped at 10 s, not 30:** a camera back from an outage is watched again within seconds.
- **Keypoints on every second frame.** The first benchmark gave eight cameras only 7.7 frames a second, with the keypoint model half the cost. Running it on 1 frame in 2 brought them to 10, with identical events on every test clip.

Measured results

Format (Mac M4)	mAP@50	1 cam	2 cams	4 cams	8 cams	8, first run
PyTorch FP32, GPU	0.883	10	10	10	8.0	6.1
PyTorch FP16, GPU	0.884	10	10	10	9.2	7.1
Core ML FP16, Neural Engine	0.882	10	10	10	10.0	7.7
Core ML 8-bit weights	0.883	10	10	10	9.9	7.8
ONNX FP32, CPU	0.883	10	6.1	4.4	2.5	2.0
ONNX INT8, CPU	0.859	10	10	6.0	2.9	2.2

Frames analysed per second per camera, target 10; the whole chain, not just the model. Measured with the fine-tuned model (Chapter 23) and the keypoint model on 1 frame in 2. "8, first run": the Phase 1 model with keypoints on every frame. With a camera unplugged for 15 s, the other three stayed at 10 frames a second (longest gap 0.12 s), and it was analysed again 3.7 s after it came back.



Frames analysed per camera as cameras are added (MacBook Air M4), with the keypoint model on 1 frame in 2. Grey: Core ML with keypoints on every frame, the first run.

Lessons from the build

- **One Mac watches eight cameras at the full rate.** The first run managed four (eight at 7.7 frames a second). The models were the limit, 15 ms per frame, and half of that was the keypoint model. Running it on 1 frame in 2 closed the gap.
 - **8-bit didn't pay off here.** Core ML with 8-bit weights runs no faster than FP16, and ONNX INT8 is *slower* than FP32 on a CPU: only the convolutions become 8-bit, so the model converts back and forth around every layer (616 extra steps).
 - **Accuracy survives the formats in use:** Core ML and PyTorch FP16 stay within 0.001 mAP@50 of the original. The exception is ONNX INT8 of the fine-tuned model (0.024 lower), which needs more calibration photos before it runs on a server.
-

CHAPTER 22

Phase 5 — Alerts, Storage and Dashboard

Week 4

Goal

Give confirmed events somewhere permanent to live, and give the safety officer a way to see and act on them without waiting for a phone notification.

Concepts You'll Apply

- Chapter 13 — REST APIs with FastAPI, PostgreSQL tables, decoupled alert queues

Step by Step

- Design the PostgreSQL schema: tables for cameras, zones, rules, and violations (with columns for type, camera, track ID, timestamp, confidence, a path to the saved snapshot, and a status of new / confirmed / false alarm).
- Save an annotated snapshot with each event — the frame with the relevant boxes and zone outline drawn on it — and set a retention policy that automatically deletes old images to control storage growth.
- Build alert workers on a queue, exactly as Chapter 13 described: Telegram first (fastest to set up via BotFather), then email, with WhatsApp last since its business API setup is more involved. Add retries and a per-camera rate limit so a misbehaving rule can't flood a channel.
- Build the FastAPI endpoints and a minimal dashboard: a live violation feed, filters by camera/type/date, a snapshot viewer, counts by hour and camera, and — critically — a "mark as false alarm" button, which is what generates the feedback loop from Chapter 14.

Common Pitfalls

- Calling the Telegram/email API directly from the same loop that processes video — reintroduces the blocking problem Chapter 13 warned about.
- Skipping the false-alarm button because it seems like a nice-to-have — without it, Phase 6's retraining loop has no labeled data to work from.

Done when: a violation on a test stream reaches your phone with a snapshot within a few seconds, and shows up correctly on the dashboard.

AS BUILT · Built and checked on the Mac, 26 September 2026

What was built

- **Storage:** the project's own PostgreSQL server on the Mac (from conda-forge, listening on this machine only), with tables for cameras, zones, rules, events and the alert queue. Each event keeps its annotated snapshot and the clean frame as files. Images older than 30 days are deleted, except those of false alarms, which are kept for retraining.
- **The video loop never waits:** it hands each event to a queue in memory, and a writer thread draws the snapshot and stores the event and its alerts in one transaction.
- **Alerts:** a separate worker takes alerts from the queue and sends them to Telegram with the picture (email works the same way). It retries with growing waits, fails at once on a wrong token, and after 5 alerts per camera in 10 minutes it holds the rest and sends one summary.
- **The dashboard** (FastAPI and one plain page) shows live camera pictures, counts, events per hour and per camera, and the event feed. Each event opens with its picture and the verdict buttons: Confirm violation, False alarm.

What changed from the plan, and why

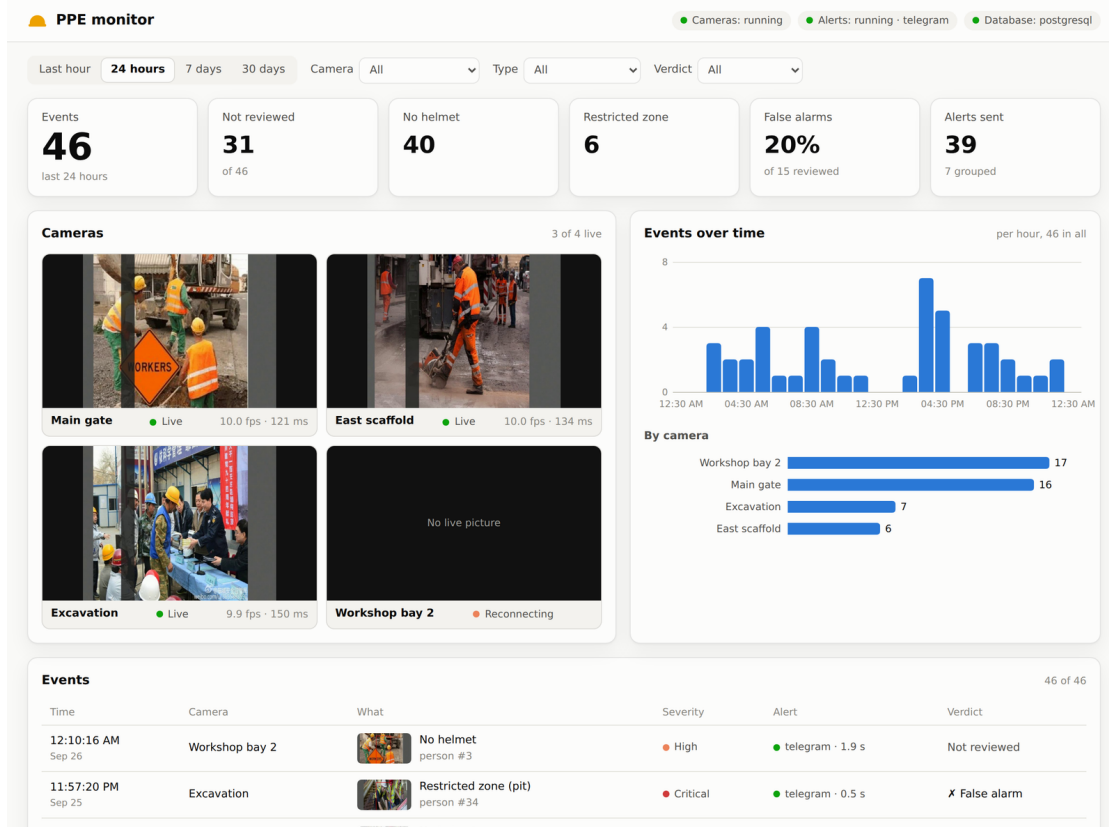
- **The queue is a database table, not Redis:** one less service to run. It survives a restart, and each alert's fate shows beside its event.
- **WhatsApp isn't built:** its business API needs an approved account, which is why the book puts it last.
- **The page asks for updates every 3 s** rather than keeping a live connection: simpler, and enough at this scale.
- **Telegram is reached over IPv4 only.** The Mac's network announces IPv6 but doesn't carry it (see the lessons).

Measured results

Measured on the Mac	Result
End-to-end check: 4 cameras, a stand-in Telegram that fails at first	all 7 checks pass

Measured on the Mac	Result
Cameras with storage and alerts switched on	10.0 frames/s each, as before
Events stored, with both images	5 of 5; none dropped
Alert to the stand-in Telegram	0.2 s after the event
Alert with the picture on the phone (real Telegram)	3.1-4.1 s (first run: 18.6-23.3 s)

The 3-4 s are 0.4 s from the event to the alert worker, then uploading the pictures, four at a time, on a connection that sends about 1 Mbit/s.



The dashboard (filled with sample events for this picture): system status, counts, live cameras, events per hour and per camera, and the event feed.

Lessons from the build

- **Test against the real service, not only a stand-in.** Alerts took 0.2 s to the stand-in and 19 s to the real Telegram. The network announced IPv6 but didn't carry it, so each new connection waited 15 s for IPv6 to time out before trying IPv4. A speed test that tries both found it in a minute.
- **Keep slow things out of the video loop.** With the database, the images and the alerts all handled elsewhere, four cameras kept their full 10 frames a second.
- **A program writing to a file may say nothing until it ends.** The dashboard's address sat in a buffer, so the start script never saw it and never opened the page.

CHAPTER 23

Phase 6 — Evaluation, Hardening and Packaging

Week 5

Goal

Turn a working prototype into something measured, resilient, and reproducible by someone other than you.

Concepts You'll Apply

- Chapter 4's five success metrics, now measured end to end rather than defined in the abstract
- Chapter 14's feedback loop, closed for the first time

Step by Step

- Annotate ground-truth violation events (start and end time) in your Phase 0 test clips, then measure event-level precision and recall, false alerts per camera per hour, and alert latency — the real, end-to-end versions of Chapter 4's metrics, not proxy numbers.
- Pull the false alarms your safety-officer dashboard button has collected, and use them as hard negatives in a retraining pass. Re-measure mAP@50 and the false-alarm rate afterward to confirm the retrain actually helped.
- Write a Docker Compose setup covering the GPU inference worker, the API, PostgreSQL, and the dashboard, with secrets kept in environment files and basic authentication in front of the dashboard.
- Write the README: an architecture diagram (Chapter 14's is a good starting point), your actual benchmark numbers, a short demo clip or GIF, and an honest "known limitations" section.

Common Pitfalls

- Treating this phase as optional polish — the benchmark numbers here are what turn a personal project into something you can defend in an interview or an audit.
- Writing a README that describes intentions rather than measured results.

Done when: *docker compose up* brings up the whole stack on a clean machine, and your README states real, measured numbers for all five headline metrics from Chapter 4.

Short on time (about four weeks)? Drop WhatsApp alerts, INT8 quantization, and the retraining loop — the system still works end to end without them. **Looking for a stretch goal?** Add detection for gloves, goggles or a safety harness, or deploy the optimized model to a Jetson-class edge device instead of a server GPU.

AS BUILT · Built and checked on the Mac and in Docker, 26 September 2026; real video from six other sites, 28 September; caps and hats, and the project complete, 29 September

What was built

- **Ground truth for every test clip:** each real violation as an episode, with its start, its end and where the person is. An episode is *sustained* (an alert is expected), *not sustained* (too short or hidden: an alert on it counts neither way), or a *compliant* person (an alert on them is false). The generated clips take theirs from the photos' labels. The two workshop clips were annotated by hand, and a small tool marks new clips.
- **An end-to-end evaluation.** Every clip plays once as a camera, eight at a time, through the real camera service, database, alert worker and Telegram's HTTP API. The API is a stand-in on the Mac, plus a few real messages to time delivery. Each alert is matched to the truth by time and place. Latency runs from the moment a violation begins on camera to the alert, and every rate comes with a 95% confidence interval.
- **The feedback loop** from the false-alarm button to a new model. Each false alarm's frame is labelled with what the model finds, plus the lesson the verdict gives: the helmet it missed, found again at low confidence. A person reviews it, and it is added to training only. The new model must be no worse on the test photos and raise no more false alarms end to end. Test footage is never used.
- **The system watches itself:** a camera down for a minute, or a camera service that stops reporting, raises its own alert, and another when it is back.
- **Docker Compose** for the whole system: PostgreSQL, the camera service, the alert worker, the dashboard (always behind a password) and demo RTSP cameras. Passwords are made on first start and every service has a healthcheck. A script checks it all from a clean copy.

What changed from the plan, and why

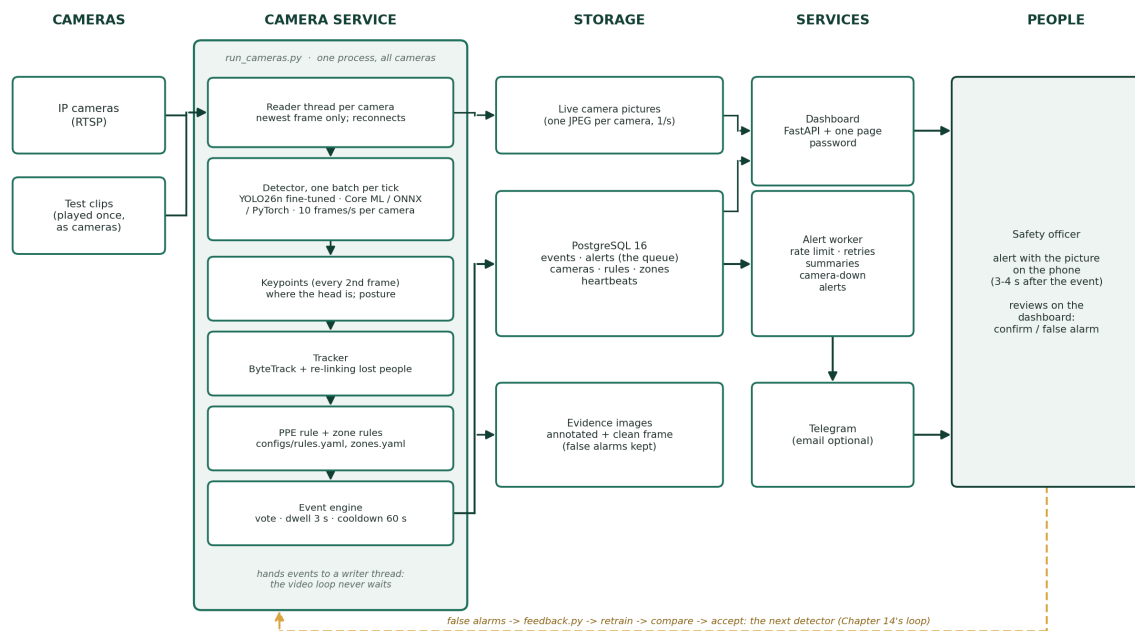
- **A CPU image, not a "GPU inference worker".** No NVIDIA GPU was available, and Docker on a Mac can't use the Apple GPU. A GPU override is written but untested. On the Mac, the native setup stays the fast one.
- **The retraining loop is built but hasn't run.** Every false alarm so far came from test clips, and training on them would make the test meaningless. It needs footage from the site that is kept out of testing.

- **Mostly generated test clips.** Of 260 clips, 258 are made from labelled public photos. Only two, 27 seconds in all, were real video. Six fixed-camera clips from other sites (free stock video, 1.2 minutes) were added afterwards and annotated by hand.

Measured results

Chapter 4's metric	Target	Measured on the Mac
mAP@50	≥ 0.85	0.883 (held-out test photos)
Throughput	8 cameras at 10 fps	9.9 fps each, 8 cameras, storage and alerts on
Alert latency	≤ 5 s	4.0 s median leaving the Mac; on the phone 7.1 s with the picture, 5.0 s without
Violations caught (recall)	$\geq 90\%$	83% (93 of 112; 95% CI 75–89%)
Alerts that are real (precision)	$\geq 90\%$	89% (93 of 105; 95% CI 81–93%)
Docker Compose from a clean copy	all up	every check passed , healthy in 24 s
Real video from six other sites	$\geq 90\%$ / $\geq 90\%$	4 of 6 violations caught; 4 of 5 alerts real (far too few to judge by)

260 test clips (43.5 minutes of video) played as cameras, eight at a time; rules as configured at the site (the vest rule off). Alerts on the phone are slower with the picture because this connection uploads about 1 Mbit/s. Without it, sending still takes 1.5 s after the event, likely because the worker waits for every alert it is sending before it takes new ones. The Docker check ran on a 2-core Linux machine.



The system as built. The dashed line is the feedback loop: false alarms marked on the dashboard become training data for the next detector.

What the misses and false alarms were

- **Six of the 12 false alarms are cyclists in bicycle helmets**, in one public photo that appears in two clip sets. The photo's labels call them helmets; the detector learnt hard hats. Two more come from a photo stored sideways, two from a small worker far behind a closer one, one from a close-up with the helmet cut off. One label looks wrong.
- **The ten missed helmets** that were looked at are mostly close-ups of faces (two of them children) and people seen from behind in the dark. None were small or cut off.
- **The nine missed zone intrusions** are the same nine as in Phase 3: small distant people, groups boxed as one, and feet cut off by the frame's bottom edge.
- **The two workshop clips raised nothing, which is right.** The two bare-headed men are in view for under 3 seconds. That is too little footage to state a false-alarm rate for the site.

Real video from six other sites

With no real violation in the workshop footage, ten free construction clips were taken from a stock-video site (Pexels). Four were dropped because the camera moves. The six kept are fixed cameras: low and high, close and far, 1.2 minutes in all. Every person near enough to judge was checked by eye, giving six real “no helmet” violations in three clips. They are test footage only. Played with the two workshop clips as eight cameras, at 10 frames a second each:

- **Four of the six violations were caught**, about 4 seconds after they began.
- **A cap passed for a helmet.** Of two bricklayers in baseball caps, one was never alerted: the model took his cap for a helmet. Few training photos show people in caps labelled as having no helmet.
- **A small worker was missed:** bent over, about 90 pixels tall, like the distant people Phase 3 missed.
- **A tractor was taken for a person.** At a frame's edge, it was followed for three seconds and raised the one false alarm. The feedback loop's instructions now say what to do with such a frame: delete the “person”, rather than draw a helmet.

Caps and hats are not helmets (29 September)

The cap was the miss most likely to recur. On photos of people in hats, the model then in use judged 27% of them “helmet worn”; its training data had hard hats and bare heads, and almost no other headwear. So 1,043 photos of people wearing hats were added to training, from Google's Open Images dataset (Flickr photos under a Creative Commons licence, each credited). A hat gets no label, so to the detector it is background: that is the lesson. The labels were checked by two models and by eye. Every hat the model half-believed was a helmet (1,939 crops) was looked at, and 63 photos turned out to show a real helmet labelled as a hat. Held-out photos are split by photographer, so no test photo shows a scene the model trained on.

	Before	Attempt 1	Attempt 2 (in use)
People in hats judged “helmet worn” (held-out photos)	27%	6.4%	2.5%
Helmets found at the alert threshold, blue	91%	86%	90%
All test clips: caught (of 118) / false alarms	94-97 / 12-13	97 / 12	98 / 12
Real video: caught (of 6)	4	5	5
Verdict		rejected: a navy helmet on the user's own clip missed	accepted, one check overridden

- **Attempt 1 used the library's default fine-tuning**, whose warm-up starts the biases at a high learning rate. It fixed the hats but lowered every helmet's score: a navy helmet in the user's workshop clip was missed, which is exactly what the September fine-tuning had fixed. Three checks failed, and a check on every helmet colour was added.
- **Attempt 2 trained gently**, a seventh of the learning rate with no bias warm-up, on fewer photos the model already got right. Losses fell steadily for 12 epochs; the last three began to memorise and were not kept. No overfitting: the gap between training and validation was 0.078, less than the old model's.
- **It passed 9 of 10 checks.** On the Phase 1 test photos its helmet score fell 0.012, against a limit of 0.01: 5 small or half-hidden helmets of 350 lost at the alert threshold, and 2 gained. The user accepted it, and the reason is recorded in the report and the model card.

Lessons from the build

- **Measure the whole path, against the clip's own clock.** Each part had been checked alone; only end to end did it show where the seconds go: 3.6 to decide, 0.4 to leave the machine, and 3 more to upload a picture.
- **Report the uncertainty with the number.** 83% from 112 violations could be anything from 75% to 89%. Even a system that caught 95% would need about 200 violations before the low end of its interval cleared 90%.
- **A test set built from photos flatters and punishes in the wrong places.** Bicycle helmets, children and sideways photos made most of the errors; none of them is what a site camera sees. Real annotated footage is the next measurement, not a nicer number.
- **Automatic labels need a person.** The first dry run of the feedback loop took a bystander's helmet for the alerted man's own. A size check and a review page were added.
- **Real video from elsewhere brings different mistakes.** The photo clips' errors were bicycle helmets, close-ups and dark backs; six real clips brought a cap, a small worker and a machine. Six violations are too few for a number (recall anywhere from 30% to 90%), but enough to show which errors a site will bring.

- **Fine-tune a trained model gently.** The defaults are made for training from a general model. On one already trained for the task they knocked the helmet scores down in the first epoch; a small learning rate and no bias warm-up kept what it knew.
- **A narrowly failed check is a decision for a person, with the reason written down.** Moving the limit after seeing the result would have hidden it; accepting it by name, with what was lost, keeps the record honest.
- **Packaging finds its own bugs.** The container's ffmpeg crashed on a host name, and a library installed itself from the internet at first start. Both were invisible on the development machine.

AS BUILT · Brought forward: the first real clips and a fine-tuned model, 25 September 2026

What was built

- **The first real footage:** two short phone videos from a fabrication workshop, kept private and used only for testing. The whole chain ran on them, and every tracked person's head was checked by eye, frame by frame.
- **A fine-tuned detector.** The Phase 1 model was trained further on three more public datasets: CHV (helmets in four colours, hi-vis vests), GDUT-HWD (18,893 helmets in four colours) and part of SH17 (indoor factories). That makes 5,051 training images, up from 2,170. Labels were checked before training, and the new model had to beat the old one before it could be used.

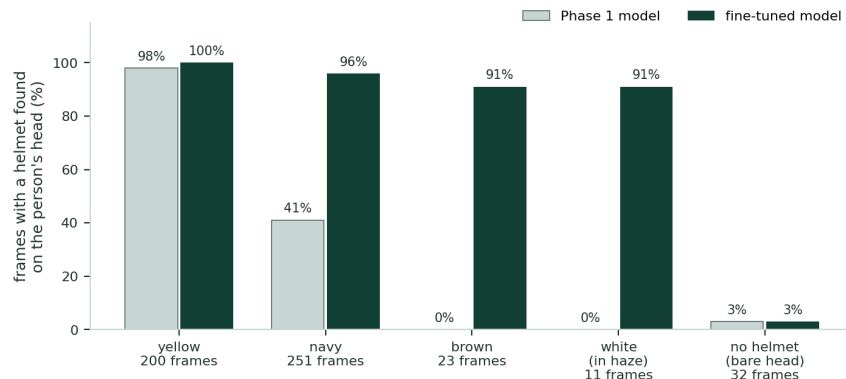
What changed from the plan, and why

- **Evaluation on real footage came early,** as soon as clips were available, instead of waiting for Phase 6. **Retraining came early too,** because of what it found.
- **Public data first, own footage second.** The book's retraining loop uses false alarms collected from the dashboard (Phase 5). Without a dashboard yet, the fastest fix was more public data with the missing helmet colours.

Measured results

Measured on photos neither model trained on	Phase 1 model	Fine-tuned
Phase 1 test photos, mAP@50	0.869	0.883
Extra datasets' test photos (428), helmet AP@50	0.666	0.918
Helmets found: blue / red / white / yellow	58 / 53 / 59 / 63%	91 / 89 / 89 / 88%
PPE rule: helmet wearer judged "missing"	4.5%	2.7%
PPE rule: bare head judged "worn"	2.3%	2.4%
Moving clips: missing helmets caught exactly once	42 / 47	40 / 47
Moving clips: false "no vest" alerts on vest wearers	6 of 81	11 of 81
Workshop clips: false helmet alerts	1	1 (a different person)

The public test photos come from the same datasets as the training photos but were never trained on. Helmets by colour are counted at the rule's threshold (0.25). The workshop clips are the two phone videos.



The workshop clips: how often a helmet was found on each person's head, grouped by what they really wore, before and after fine-tuning.

What the real clips showed first

- **With the Phase 1 model, yellow helmets were found almost every time; navy, brown, and white ones in haze mostly weren't.** The one "no helmet" alert was a false alarm on a man wearing a navy helmet throughout. The two men who really had no helmet were judged correctly.
- **Why:** the training photos are mostly outdoors, with yellow, white and orange helmets. Dark helmets seen from behind against bright skylights hardly appear. This is Phase 1's failure case, met in real life.
- **No one at that site wears hi-vis**, so the vest rule alerted on every worker. Hi-vis isn't required there, so that rule is now off: rules must follow each site's policy.
- **A hand-held phone confuses the tracker**, which expects a fixed camera. The next test clips should be filmed from a fixed, high spot, like CCTV.

How the retraining guards against overtraining

- It starts from the Phase 1 model rather than from scratch, runs at most 30 epochs, and stops by itself after 8 without improvement on photos it doesn't train on. The version kept is the best one on those photos, not the last one, and a report compares training with validation, epoch by epoch.
- The new model replaces the old one only if it is no worse on the original test photos and better on the new ones. Then it is checked again on the workshop clips.

The overfitting check, and why the model was accepted anyway

Training ran all 30 epochs in 193 minutes on the Mac; the version kept is from epoch 24. The report flagged it, by a small margin. It scores 0.085 mAP@50 higher on its own training photos than on validation photos, and the line had been set at 0.08 before training began. Everything else pointed the other way. The validation loss fell for all 30 epochs and never turned upwards. The validation score stayed flat for the last seven epochs rather

than falling. And its mAP@50 rose on every set of test photos.

Most of the gap comes from the last ten epochs, which Ultralytics trains without mosaic. The training photos get easier, so the model scores better on them: the training loss drops sharply at epoch 21 while the validation loss doesn't move. A model that memorises gets worse on new photos; this one got better. So it was accepted, and the line stays at 0.08 for the next round. It went into use the same evening.

What is still wrong

- **One false helmet alert on the workshop clips.** The Phase 1 model's false alarm, on a navy helmet, is gone. The new one is on a man in a white helmet, about 155 pixels tall in the thickest haze. His track had jumped between several people as the phone turned, so the three seconds of "missing" were not all his.
- **Public data taught the colours, not this site.** For its haze and distances, the next step is labelled footage from a fixed camera there.
- **Vests are a trade-off** (not at this site, where the vest rule is off). The new model misses more half-hidden vests: on the moving clips, false "no vest" alerts rose from 6 to 11 of 81 wearers. The likely cause: GDUT-HWD has no vest labels, so vests the label check missed were learned as background.

Lessons from the build

- **A test set is only as good as its resemblance to the real site.** 0.869 mAP@50 on public test photos said nothing about navy helmets under skylights.
- **Look at every alert.** One false alarm, looked at closely, explained the whole problem.
- **More of the right data beat more training.** The validation score was flat after epoch 23; photos with navy helmets in them took navy helmets from 41% to 96% of frames.
- **Decide what "better" means before training, then look beyond it.** The checks were fixed before the run and all about helmets; the side effect showed up in vests, the class the new data labelled worst.

CHAPTER 24

The Study Plan: What to Learn, Week by Week

Part II taught every concept this project needs; this chapter is about pacing that learning against the build schedule in Chapters 17-23, so you're never blocked waiting to understand something. Budget roughly 10-12 hours for Week 0, then 4-5 hours of study per week alongside the build work. Each week below ends with a small, concrete exercise — do it before moving on; it's the difference between having read about an idea and being able to use it.

Week 0 — Foundations (skip what you already know)

Study: Re-read Chapters 5-6 closely. OpenCV's video I/O and drawing functions; the concept of IoU, precision, recall and mAP.

Prove it: Run a pretrained YOLO model on a video clip and save the annotated output; compute IoU for two boxes by hand, then again in code, and confirm they match.

Week 1 — Data and Training (pairs with Chapter 18 / Phase 1)

Study: Chapters 7-8 on CNNs and YOLO. Ultralytics' training documentation; the ideas of data augmentation, class imbalance, and dataset leakage.

Prove it: Fine-tune on a small PPE dataset (a free Colab GPU is enough) and write down three specific failure cases you observed.

Week 2 — Tracking, Geometry and Rules (pairs with Chapters 19-20 / Phases 2-3)

Study: Chapters 9-10. The intuition behind the Kalman filter and the Hungarian algorithm; point-in-polygon tests.

Prove it: Run tracking on a clip, plot track IDs over time, and count how many ID switches occur; write a few rules-engine unit tests using fake, hand-written tracks.

Week 3 — Streaming and Concurrency (pairs with the first half of Chapter 21 / Phase 4)

Study: Chapter 11. RTSP and H.264 basics; the difference between threads and processes in Python and what the GIL does and doesn't restrict.

Prove it: Read four fake RTSP streams at once and confirm the displayed frame never falls meaningfully behind live; kill one stream and confirm it reconnects automatically.

Week 4 — Optimization, Backend and Alerts (pairs with the rest of Chapter 21 and Chapter 22 / Phases 4-5)

Study: Chapters 12-13. ONNX export and TensorRT basics; FastAPI's tutorial; the Telegram Bot API and SMTP.

Prove it: Produce a latency table for FP32 vs. FP16 on your exported model; build one FastAPI endpoint that stores an event and sends a Telegram message.

Week 5 — Packaging and Evaluation (pairs with Chapter 23 / Phase 6)

Study: Docker and Docker Compose basics; the NVIDIA Container Toolkit if using a GPU in a container; how to measure event-level precision, recall and latency rather than just per-frame accuracy.

Prove it: Follow only your own README on a clean machine or a fresh virtual machine, and get the full stack running without improvising any step.

For the full reading list behind every chapter — documentation, the ByteTrack paper, tutorials, and the sources behind Chapter 1's statistics — see Appendix D.

PART V

Appendices

A glossary, a metrics reference, a prerequisites checklist, further reading, and answers to the questions this book gets asked most often.

APPENDIX A

Glossary of Terms

Every term below was introduced in context somewhere in Parts I-III; this is a single alphabetical place to look one up again.

Alert latency — The time from a violation starting on camera to the alert reaching a person.

Alert queue — A buffer that decouples confirming a violation from the slower work of sending a notification about it.

Bounding box — The rectangle that encloses a detected object.

ByteTrack — A tracking algorithm that also uses low-confidence detections to keep tracks alive through occlusion.

Calibration dataset — A small, representative set of real images used to guide accurate INT8 quantization.

CCTV — Closed-Circuit Television — a private camera network.

Channel — One of the separate numeric grids making up an image (3 for RGB, 1 for grayscale).

CNN (Convolutional Neural Network) — A network architecture built around small, reusable filters scanned across an image.

Codec — A method for compressing and decompressing video, e.g. H.264.

Confidence score — A 0-1 number expressing how sure a model is about a detection.

Cooldown — A minimum waiting period before the same track can trigger the same alert type again.

Data association — The problem of matching new detections to existing tracks each frame.

Dwell time — The minimum duration a condition must persist before it counts as a confirmed event.

Event — One confirmed violation by one tracked person, counted once however many frames it spans.

False positive / false alarm — A case where a system reports something that did not actually happen.

Fatal Four — OSHA's name for falls, struck-by, caught-in/between and electrocution — the leading causes of US construction deaths.

Filter / kernel — A small grid of weights that slides across an image detecting one local pattern.

FP32 / FP16 / INT8 — Progressively lower-precision number formats a model can be converted to run in.

FPS (Frames Per Second) — How many frames a video shows, or a camera captures, each second.

GIL (Global Interpreter Lock) — A Python-specific restriction preventing two threads from executing Python bytecode at the same instant.

Hungarian algorithm — An efficient method for finding the best pairing between two sets — here, tracks and detections.

Inference — Running a trained model to get predictions, as opposed to training it.

IoU (Intersection over Union) — A measure of overlap between two boxes.

JSON — A lightweight, structured text format for exchanging data between programs.

Kalman filter — An algorithm that predicts an object's next position from its previous position and velocity.

Keyframe (I-frame) — A complete, independently decodable video frame.

Layer — A group of units in a neural network that process the previous layer's output together.

Layer fusion — Merging several small computational steps into one more efficient operation.

mAP (mean Average Precision) — A single score summarizing detection accuracy across confidence levels, at a given IoU threshold.

Message queue — A buffer decoupling a fast, time-critical process from a slower one.

Missed-violation rate / false-alarm rate — The two ways an alert system can be wrong: silence during a real violation, or an alert about one that didn't happen.

Multi-object tracking (MOT) — Assigning a persistent identity to each detected object across frames.

Near-real-time — Fast enough that the response still matters — here, seconds, not minutes.

NMS (Non-Maximum Suppression) — Collapsing duplicate overlapping boxes down to one per real object.

Object detection — Finding what objects are in an image and where each one is.

Occlusion — When an object is partly or fully hidden behind something else.

ONNX — A shared, framework-independent file format for trained models.

Pixel — The smallest single element of a digital image.

Point-in-polygon test — Determining whether a point lies inside a polygon-shaped region.

PPE (Personal Protective Equipment) — Equipment worn to reduce exposure to a hazard — here, helmets and high-visibility vests.

Precision / Recall — Precision: how much of what was flagged is correct. Recall: how much of what's really there was caught.

Pretrained weights — Weights already learned from a large general dataset, used as a training starting point.

Quantization — Reducing a model's numeric precision to trade a little accuracy for speed.

Queue — A buffer holding items waiting to be processed.

Ray casting — A method for solving point-in-polygon by counting ray-edge crossings.

Resolution — An image's grid size, given as width × height.

REST API / endpoint — A standardized way for software to request an action or data over HTTP; an endpoint is one such URL.

Restricted zone — An area, drawn per camera, that people must stay out of — always or at set times.

RGB — A color model storing each pixel as red, green and blue intensity.

RTSP — Real Time Streaming Protocol — how IP cameras expose a live video feed.

Single-stage detector (YOLO) — A detector that predicts all boxes and classes in one forward pass.

SMTP — The standard protocol used to send email.

State machine — A model tracking an entity's current status and the rules for changing it.

TCP vs. UDP — Two network transport methods: TCP guarantees delivery and order; UDP is faster but can drop data.

TensorRT — An NVIDIA library that compiles a model into an optimized engine for a specific GPU.

Thread — A separate line of execution within a program.

Throughput — How much video the system keeps up with — streams × frames per second on one machine.

Track ID — The stable identifier attached to one tracked object over time.

Training / loss / epoch — Training: adjusting weights from labeled examples. Loss: the measured error. Epoch: one full pass through the training data.

Transfer learning / fine-tuning — Continuing to train a pretrained model on a smaller, task-specific dataset.

Two-stage detector — An older detection approach that proposes regions, then classifies them separately.

Vigilance decrement — The documented drop in a person's ability to detect rare events the longer they watch a mostly uneventful feed.

Weight — An adjustable number inside a neural network, learned from data.

APPENDIX B

Metrics Reference Sheet

The five numbers this project is judged on (Chapter 4), a reasonable starting target, how each is measured, and what this build measured in Phase 6 (Chapter 23: MacBook Air M4, 26 September 2026, 260 test clips played as cameras).

Metric	Suggested Target	How to Measure	Measured (Phase 6)
mAP@50 (person, helmet, vest)	0.85 or higher on a held-out test set	Validation on scenes never trained on, reported separately for your own clips	0.883 on 200 held-out test photos (0.885 on 628)
Throughput	8 streams at 10+ FPS each on one mid-range GPU; 2–4 streams at ~5 FPS on CPU only	Benchmark script logging FPS per stream at 1, 2, 4 and 8 streams	9.9 FPS per stream, 8 streams, with storage and alerts on (Apple M4, Core ML)
Alert latency	5 seconds or less from the violation starting on camera to the alert arriving, including the 2–3 s dwell time	Violation start times annotated in your test clips vs. the time each message arrives	4.0 s median leaving the machine; on the phone 7.1 s with the picture, 5.0 s without
Missed-violation rate (event recall)	90%+ of real, sustained violations produce an alert	Ground-truth violation events annotated in your test clips (Phase 6), matched against alerts	83% caught (93 of 112; 95% CI 75–89%); on real video from other sites, 4 of 6

Metric	Suggested Target	How to Measure	Measured (Phase 6)
False-alarm rate (event precision)	90%+ of alerts are real violations; also report false alerts per camera per hour	Reviewers mark alerts on the dashboard, using your test clips	89% real (93 of 105; 95% CI 81–93%); 16.6 false alerts per camera-hour, with people always in view; on real video from other sites, 4 of 5

APPENDIX C

Prerequisites Checklist

Knowledge

- Python, including virtual environments and basic threading
- OpenCV basics: reading and writing video, drawing boxes and polygons
- Deep learning basics: CNNs, train/val/test splits, overfitting, transfer learning
- Detection metrics: IoU, precision, recall, mAP
- Basic SQL, REST APIs, Git and the Linux command line
- Docker basics (helpful from the start, essential by Phase 6)

Hardware and Accounts

- An NVIDIA GPU machine for TensorRT; an entry-level card is enough. Without one, train on Colab or Kaggle, run on CPU with ONNX Runtime, and borrow a cloud GPU for the TensorRT benchmark.
- A fake CCTV setup: MediaMTX as the RTSP server, with ffmpeg looping video files into it as several cameras.
- 5–10 test clips (1–3 minutes each) of workers with and without helmets and vests, in different light and angles; check licenses or get permission.
- A Roboflow account for datasets, a Telegram bot token from BotFather, and an email account with SMTP or app-password access.
- WhatsApp needs a business API (Twilio sandbox or Meta's WhatsApp Cloud API) with more setup, so leave it for last.

Software

Python 3.10+, PyTorch, Ultralytics YOLO, OpenCV, ONNX Runtime, TensorRT with matching CUDA/cuDNN and driver, FastAPI, PostgreSQL, Docker with the NVIDIA Container Toolkit, ffmpeg.

Ready-to-start check: you can (1) run a pretrained YOLO model on a video and save the annotated output, (2) explain IoU and mAP@50 in your own words, and (3) write a small REST endpoint. If not, spend Week 0 of the study plan (Chapter 24) closing those gaps first.

Licensing note: Ultralytics YOLO is released under AGPL-3.0, with a separate commercial license available. That's fine for a portfolio project, but check it before shipping in a commercial product.

APPENDIX D

Further Reading

- **Ultralytics documentation** — the train, val, track and export pages, plus the performance-metrics guide.
 - **Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box" (2021)** — the paper behind Chapter 9's tracker.
 - **OpenCV official tutorials** — video I/O, drawing, and geometric operations.
 - **ONNX Runtime and NVIDIA TensorRT documentation** — export formats and engine building.
 - **The FastAPI tutorial** — the fastest path to a working REST API in Python.
 - **The PostgreSQL tutorial** — schema design and everyday SQL.
 - **Docker's "Get Started" guide**, plus the NVIDIA Container Toolkit docs for GPU access inside containers.
 - **The MediaMTX README** — for the fake-camera RTSP setup used from Phase 0 onward.
 - **The Telegram Bot API documentation** — for the first (and simplest) alert channel.
 - **Stanford CS231n** — for readers who want the fuller mathematics behind CNNs and object detection, referenced in Chapter 7.
 - **International Labour Organization**, "*Nearly 3 million people die from work-related accidents and diseases*" (2023) — the source for Chapter 1's global figures.
 - **U.S. Bureau of Labor Statistics and OSHA** — 2023 construction fatality data and the Fatal Four hazard categories, the sources for Chapter 1's US figures.
-

APPENDIX E

Frequently Asked Questions

Do I need a GPU to start?

No. Phase 0 and much of Phase 1 work fine on CPU or a free Colab GPU. A GPU becomes important starting in Phase 4, when the goal shifts to running several live streams at once — and even then, ONNX Runtime on CPU is a legitimate, slower fallback that still lets you

finish and demonstrate the whole system.

Which YOLO version should I use?

This book deliberately doesn't pin one, because Ultralytics updates its recommended version regularly and the concepts in Chapter 8 apply to all of them. Start with whichever the Ultralytics documentation currently recommends as its small, general-purpose model, and treat the exact version as a Phase 1 implementation detail, not a design decision.

How much training data do I actually need?

Thanks to transfer learning (Chapter 8), fine-tuning from pretrained weights needs far less data than training from scratch — a few hundred to a couple thousand well-labeled images per class is a reasonable starting point, expanded later with hard negatives from your own false-alarm review loop (Chapter 14).

My false-alarm rate is too high. What's the first thing to check?

In order: first, confirm your PPE-matching geometry (Chapter 10) against your own site's camera angles specifically, not just the training dataset's; second, check whether dwell time and cooldown are tuned for your actual scene (Chapter 19); third, look at whether the false alarms cluster around one specific lighting condition or camera, which usually points to a data gap in Phase 1 rather than a logic bug.

Can this run entirely on CPU, with no GPU at all?

Yes, with reduced throughput — expect roughly 2–4 streams at around 5 FPS each on a modern multi-core CPU using ONNX Runtime, versus 8 or more streams at 10+ FPS on a mid-range GPU with TensorRT (Chapter 21's benchmarking step measures your specific hardware directly rather than relying on this rule of thumb). **Measured in this build:** on a MacBook Air M4, ONNX Runtime on the CPU handled one camera at 10 frames a second but only about 16 frames a second in total across several, while the same laptop's Neural Engine (Core ML) handled eight cameras at 10 frames a second each, with the keypoint model on every second frame. A 2-core cloud server with no GPU managed 6 frames a second for a single camera.

Is it legal or privacy-compliant to deploy this on a real site?

That depends entirely on your jurisdiction and your employer's policies around workplace video monitoring and worker consent, and this book doesn't attempt to give legal advice on it. Note that the system as designed tracks anonymous track IDs, not personal identities — it never performs facial recognition or attempts to identify who a specific worker is —

which meaningfully reduces (but does not eliminate) the privacy questions involved. Check with whoever handles compliance at your organization before any real deployment.

How is this different from just buying a commercial PPE-detection product?

A commercial product will likely detect PPE violations out of the box with less effort. Building this yourself buys three things a purchased product can't: a system tuned specifically to your own cameras and site conditions rather than a vendor's generic dataset, full control over the false-alarm-review and retraining loop, and — if you're building this as a learning project rather than a production deployment — direct, defensible proof that you can design and ship a real-time computer vision system end to end.

What's the single hardest part of this project, honestly?

Not the machine learning — fine-tuning YOLO (Phase 1) is usually the most approachable part for newcomers, since Ultralytics does most of the heavy lifting. The genuinely hard part is almost always Phase 4: making several live streams behave reliably, at speed, on hardware that wasn't built for it, without one misbehaving camera taking the rest down with it. Budget your time accordingly.

APPENDIX F

Build Log

The dated record of the build. Each phase is built, tested on the Mac, and reviewed before the next begins; the numbers here are the measured ones from each phase's As Built notes.

Date	Milestone	Key result
20 Sep 2026	Project plan, and the first edition of this book	—
21 Sep 2026	Phase 0 verified on the Mac (Ch. 17)	3 fake cameras at 15 frames/s; Apple GPU 6.7× the CPU
23 Sep 2026	Phase 1 reviewed (Ch. 18)	Detector mAP@50 0.869 on the held-out test photos
24 Sep 2026	Phase 2 reviewed, with walking, bending and crouching workers (Ch. 19)	42 of 47 missing helmets caught exactly once on moving clips
24 Sep 2026	Phase 3 checked on the Mac and reviewed (Ch. 20)	27 of 37 zone intrusions caught once; 0 false
25 Sep 2026	Phase 4 checked on the Mac (Ch. 21)	4 cameras at 10 frames/s, 8 at 7.7; a camera drop-out survived

Date	Milestone	Key result
25 Sep 2026	First real clips from a workshop (Ch. 23)	Navy helmets found in 41% of frames; 1 false alert
25 Sep 2026	Dataset extended with three more public datasets (Ch. 23)	5,051 training images, up from 2,170
25 Sep 2026	Fine-tuned model passed every check and went into use (Ch. 23)	Navy helmets found in 96% of workshop frames (was 41%); test mAP@50 0.883
25 Sep 2026	Phase 4 done: keypoint model on 1 frame in 2 (Ch. 21)	8 cameras at 10 frames/s on one Mac; same events on every test clip
26 Sep 2026	Phase 5 checked on the Mac (Ch. 22)	Alert with the picture on the phone 3–4 s after the event; cameras still at 10 frames/s
26 Sep 2026	Phase 6 checked on the Mac (Ch. 23): 260 test clips end to end	mAP@50 0.883; 8 cameras at 9.9 frames/s; alert leaves in 4.0 s; 83% caught; 89% of alerts real
26 Sep 2026	Docker Compose: the whole system from a clean copy (Ch. 23)	Every check passed; healthy 24 s after start
28 Sep 2026	Real video from six other sites, end to end (Ch. 23)	4 of 6 violations caught, 4 of 5 alerts real; misses: a cap, a small worker; false: a tractor
28 Sep 2026	Fifth edition of this book	All six phases built
29 Sep 2026	Caps and hats: attempt 1 rejected (Ch. 23)	Hats judged helmets 27% -> 6.4%, but every helmet colour ~5 points lower; a navy helmet missed
29 Sep 2026	Caps and hats: attempt 2 in use (Ch. 23)	Hats judged helmets 27% -> 2.5%; 9 of 10 checks passed; one accepted by the user, reason recorded
29 Sep 2026	Project complete: sixth edition, the user guide (Appendix G)	Everyday commands in one script; real cameras need only their address; photo and video checks
30 Sep 2026	The PPE Monitor app (Appendix G)	Double-click to start; any part that stops is started again; optional start at log-in
7 Oct 2026	Ready for GitHub: a private repository, pushed by one script	Code AGPL-3.0; the model non-commercial; clips, data, logs and secrets stay on the Mac

APPENDIX G

Using the System

The system as built, from the operator's side. Every command runs in the Terminal on the Mac; `bash scripts/ppe.sh help` lists them. The full guide, with examples, is `docs/USER_GUIDE.md` in the project.

Everyday commands

Command (bash scripts/ppe.sh ...)	What it does
app [--login]	Make the PPE Monitor app (Applications and the Desktop): double-click to start the monitor, which restarts any part that stops and keeps the Mac awake, or to open the dashboard if it is running. --login starts it at log-in.
start	Watch every camera in configs/cameras.yaml: alerts to the phone, the dashboard at 127.0.0.1:8080. Ctrl+C stops.
photo PHOTO	Who wears a helmet in a photo: red, green or grey boxes, written next to it
video VIDEO [CAMERA]	Check a video file, with a camera's zones and rules: an annotated copy and the events
camera CAMERA	One camera in a window, to check its address and view
zones CAMERA / rules	Draw a camera's restricted zones; check the rules and zones files
telegram	Connect alerts to Telegram (once)
check / evaluate	Is everything working (3 min); the five headline numbers on every test clip (7 min)
feedback / retrain	False alarms marked on the dashboard become training data; a new model, accepted only if it passes every check
model / stop	Which model is in use and how it was accepted; stop the database server

Adding a real camera

- Find its RTSP sub-stream address (about 720p). Hikvision: `rtsp://USER:PASS@IP:554/Streaming/Channels/102`; Dahua and CP Plus: `.../cam/realmonitor?channel=1&subtype=1`.
- Put its password in `configs/secrets.env` (e.g. `CAM_GATE_PASSWORD=...`) and the camera in `configs/cameras.yaml` with `${CAM_GATE_PASSWORD}` in its address, without `sim_source`. Check it with `ppe.sh camera gate`.
- **Place it high, looking down, in landscape**, so that people are at least a sixth of the picture's height: smaller people are not judged. Keep a restricted zone, and the ground people stand on, wholly in view.
- Draw its zones, add their rules to `configs/rules.yaml`, check them, then `ppe.sh start`.

Keeping it accurate

- **Mark every false alarm** on the dashboard. Its clean picture is kept; with ten or more from real cameras, feedback and retrain teach the next model, which must pass every check before `mac_phase6.sh accept` puts it into use.
- **Measure it on your own site:** record a few minutes from your fixed cameras, mark the violations (`mac_phase6.sh annotate`), and run `evaluate`. This is the one number the project still lacks.

- **Go back to an older model** by making it the newest file in `models/` (touch it) and naming it in `configs/ppe.yaml`.

The model on its own

`models/ppe4caps_yolo26n_caps2.pt` is an Ultralytics YOLO26n with three classes: 0 person, 1 helmet, 2 vest, at an input of 640 pixels. The system's thresholds are person 0.40, helmet 0.25 and vest 0.25; Core ML and ONNX versions are in `models/exported/`. The detector finds helmets; whether a person *wears* one is the rule's decision (Chapter 10), which `PPEMonitor.process()` applies to each frame, exactly as the cameras do.

The detector alone (the project's environment)

```
from ultralytics import YOLO
model = YOLO("models/ppe4caps_yolo26n_caps2.pt")
r = model.predict("photo.jpg", imgsz=640, conf=0.25)[0]
for cls, conf in zip(r.boxes.cls.tolist(), r.boxes.conf.tolist()):
    print(model.names[int(cls)], round(conf, 2))
```